

BlueLM-GUI Technical Report: A Real-Device-Centric Flywheel for Self-Improving Mobile GUI Agents

vivo AI Lab

<https://github.com/vivo-ai-lab/BlueLM-GUI>

Abstract. Mobile GUI agents are shifting from multi-module frameworks to native models trained end-to-end, yet industrial deployment faces three persistent gaps. Sandbox training produces a distribution mismatch with production environments; expensive real-device failures remain underutilized; and fixed benchmarks saturate, losing the power to guide iteration. We present **BlueLM-GUI**, a 35B-A3B mobile GUI agent built as a real-device-centric flywheel that closes these gaps through three principles. *Every Sample Matters*: a dual-track pipeline with Heterogeneous Triple-System Consensus evaluation and an Error Correction & Derivation Module salvages every trajectory into usable supervision. *Every Rollout Is Real*: a three-stage recipe—continual pre-training, supervised fine-tuning, and agentic reinforcement learning on hundreds of real phones—grounds every rollout in real production environments, so the capability the model learns transfers directly to deployment. *Every Query Evolves*: a quota-driven benchmark methodology with three orthogonal axes enables precise attribution and allows the benchmark to be systematically upgraded as the model improves. BlueLM-GUI achieves **87.4** on MobileGUI-VBench, surpassing the best closed-source model by 5.1 points, and **84.9** on AndroidWorld, the best result among open-source models and competitive with closed-source models. These results demonstrate that grounding model training and iterative improvement in both real devices and the three Every principles yields strong, robust, and transferable mobile GUI capability.

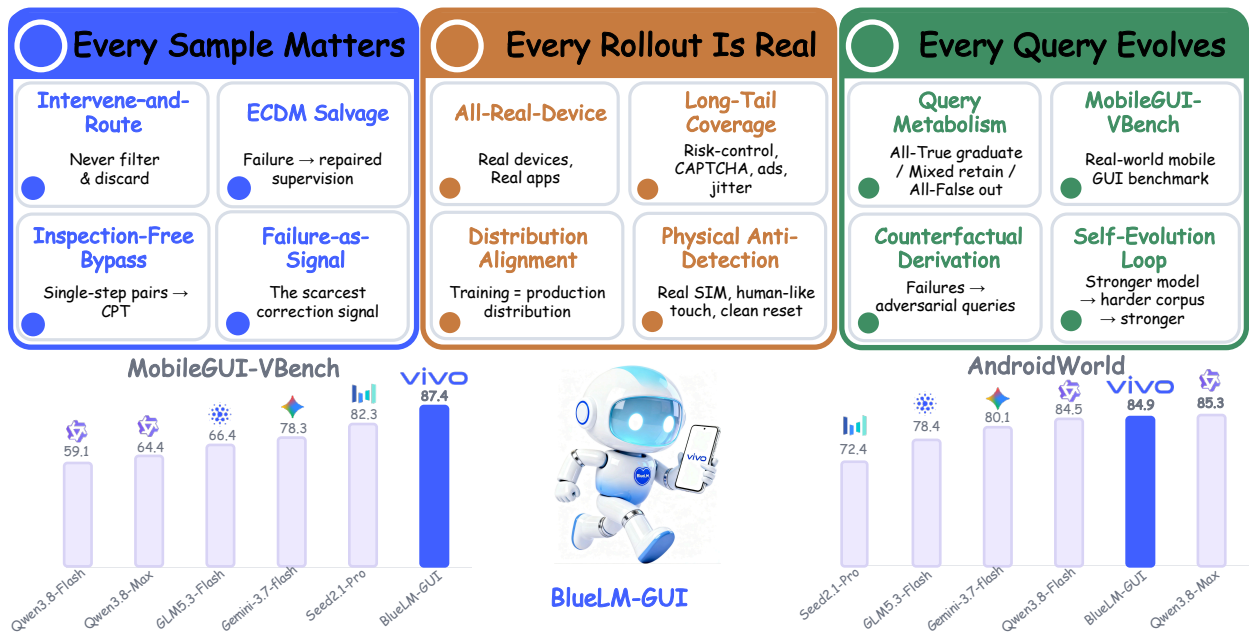


Figure 1 | Overview of BlueLM-GUI, a real-device-centric flywheel built around three principles.

Contents

1	Introduction	4
2	Training Data Construction	6
2.1	Data System Overview	6
2.1.1	Design Philosophy	6
2.1.2	CPT/SFT Decoupling	7
2.1.3	Dual-Track Pipeline	7
2.2	Query Generation	8
2.2.1	Function Tree	8
2.2.2	Instruction Tiers	8
2.2.3	User Personas	9
2.2.4	Capability Profile	10
2.2.5	Global Environment Memory	11
2.2.6	Query Intake Processing	12
2.3	All-Real-Device Trajectory Collection	13
2.3.1	Trajectory Collection Methods Overview	13
2.3.2	Semi-Automatic Collection	14
2.3.3	Automatic Collection	14
2.3.4	Schema-Guided Collection	14
2.3.5	Auxiliary External Query Channel	15
2.3.6	Structured CoT	15
2.3.7	Horizontal Enrichment	16
2.4	Trajectory Evaluation and Data Routing	16
2.4.1	Two-Phase Evolution of Data Routing	17
2.4.2	Heterogeneous Triple-System Consensus	17
2.4.3	Data Pool Dynamics	19
2.5	The Self-Evolution Loop	19
2.5.1	Cascaded Failure Attribution	20
2.5.2	Error Correction & Derivation Module	21
2.5.3	Lifecycle of the Self-Evolution Loop	23
3	Model Training	23
3.1	Continual Pre-training	24
3.1.1	Training Corpus and Objective	24
3.1.2	Interaction Protocol	24
3.2	Supervised Fine-tuning	25
3.2.1	Intent-Level Actions	26
3.2.2	Behavioral Boundaries	26
3.3	Agentic Reinforcement Learning	26
3.3.1	Real-Device Environment	27
3.3.2	Reward Design	27
3.3.3	Training Optimization	27
4	MobileGUI-VBench	28
4.1	A Quantified Metric System	29
4.1.1	Scenario Metric	29
4.1.2	Complexity Metric	29
4.1.3	Interaction and Risk Metric	29
4.2	Benchmark Construction	30
4.2.1	Scenario Quotas	30
4.2.2	Complexity Quotas	30

4.2.3	Interaction and Risk Quotas	31
4.2.4	From Quotas to Queries	31
4.3	Evaluation Protocol	31
5	Experiments	32
5.1	Training Details	32
5.2	Comparison with State-of-the-Art Models	33
5.3	Ablation Studies	34
5.3.1	The Necessity of Continual Pre-training	34
5.3.2	The Form of the Interaction Context	35
5.3.3	The Data Mixture across Stages	36
5.3.4	The Role of the Real-Device Online Environment	36
5.3.5	The Reliability of the Reward Signal	37
6	Related Work	37
7	Conclusion	39
8	Contributors	40
A	Appendix	46

1. Introduction

Recent advances in Large Language Models (LLMs) (Anthropic, 2026; DeepSeek-AI et al., 2026; Google DeepMind, 2026; OpenAI, 2026) have approached or surpassed human-expert performance in natural language tasks (Kimi Team et al., 2026; OpenAI, 2026) and coding (Merrill et al., 2026; Zhuo et al., 2025), and across a wide range of downstream domains. This capability is rapidly extending along a curve from pure-text interaction toward vision-language-action multimodal agents. On this curve, mobile GUI agents occupy a uniquely critical position: mobile devices concentrate the largest and most frequently used entry points to digital services, where task-automation demand is explicit and commercial value is direct, making GUI agents one of the most industrially valuable directions among multimodal agents. Early systems were typically built as multi-module frameworks, composing planner, executor, and memory modules around general-purpose VLMs (Wang et al., 2024a; Yang et al., 2023); however, error propagation and information loss between modules cascade with task length, and multi-round module calls introduce unacceptable latency. Leading native GUI agents have therefore shifted to a different route: integrating perception, grounding, planning, and execution into a single model through post-training, allowing the model to learn decisions directly in an end-to-end closed loop (Cao et al., 2026; MAI-UI Team, 2026; Shen et al., 2026; Tencent Hunyuan Team, 2026).

As the industry’s GUI agent technology converges on the single-model paradigm (ByteDance Seed, 2025b; MAI-UI Team, 2026; Qin et al., 2025a; Ye et al., 2025), the focus of system design shifts from module engineering to representation learning inside the model—and the latter depends largely on the quality of real agent data. This shift implies that what limits a mobile GUI agent today is no longer architectural sophistication, but problems that lie elsewhere: whether the training distribution matches the real distribution of production environments, and whether the benchmark can still accurately locate capability gaps after each iteration. The former concerns the data engine; the latter concerns the evaluation methodology. Yet in industrial practice, both remain fundamentally unsolved: the environments that generate training data are distributionally inconsistent with deployment (Liu et al., 2026a; Lu et al., 2024; Rawles et al., 2025; Wu et al., 2026; Xu et al., 2024), the trajectories that carry the strongest signals remain under-utilized (Hou et al., 2026; Lian et al., 2026; Wang et al., 2026b), and the benchmarks that steer iteration quietly saturate (Kong et al., 2026; Li et al., 2026; Sui et al., 2026). What truly differentiates leading systems is how fast they close—and keep turning—the loop of “data production-training-evaluation-attribution-reproduction.”

Concretely, these problems manifest in practice as three gaps that stand in the way of closing this loop. The first is a distribution gap: the dominant approach trains in sandbox or emulator environments, or pairs them with real devices (Cao et al., 2026; Liu et al., 2026a; Lu et al., 2024; Rawles et al., 2025; Shen et al., 2026; Tencent Hunyuan Team, 2026; Wu et al., 2026; Xu et al., 2024; Zhou et al., 2025). Sandboxes offer resettability and controllability, yet their environment composition, application states, and user-behavior distributions exhibit a substantial shift from real production environments and real usage. The second is a utilization gap: collecting trajectories on real devices is expensive and slow, and the resulting data contains a large share of failed or anomalous trajectories; these carry the scarcest correction and exploration signals, yet remain under-utilized (Chen et al., 2025; Wu et al., 2025). The third is an attribution gap: under a self-evolution paradigm the model keeps improving, so any fixed benchmark saturates over time and loses its diagnostic power (Ye et al., 2026; Zhu et al., 2026)—once a benchmark no longer separates good decisions from bad ones, its score stops telling us what to fix next. What is needed is not a better benchmark, but a *method for producing benchmarks* that evolves together with the model. Motivated by these three gaps, we present **BlueLM-GUI**, a mobile GUI agent built as a real-device-centric flywheel for self-improving capabilities. We design data collection, model training, and evaluation as one closed loop governed by three complementary principles, one at each level of the system. We elaborate each principle in

turn, following the pipeline they govern—data, training, and evaluation.

Every Sample Matters. Every collected trajectory is put to use—a direct response to the utilization gap above: failed and anomalous trajectories carry the rarest learning signal of all, so letting them slip away is a waste no system can afford. Successful trajectories enter the SFT pool as gold-standard supervision; failed or environmentally anomalous ones are routed to the Error Correction & Derivation Module, which salvages them into new supervision through step-level correction, retroactive query alignment, or counterfactual query derivation. This principle governs the entire collection-and-training pipeline on our all-real-device farm: whether produced by guided collection or RL self-evolution, every trajectory is triaged by value—usable ones enter the pool, flawed ones are repaired and reborn, and those with no further incremental value retire naturally.

Every Rollout Is Real. This is our answer to the distribution gap. For the training distribution to align with deployment, the most thorough way is to let training itself happen in the deployment environment—all of our RL exploration runs on real devices: every rollout is a real interaction within a real environment, whose state transitions, interface feedback, and anomalous events all come from the production distribution, so the capability the model acquires naturally fits deployment—a source of robustness that no simulator can offer. After each rollout, the trajectory flows back into the data pool: verifiable successes become supervision for later rounds, and failures tell us which tasks to sample next and thus the flywheel turns.

Every Query Evolves. This is our answer to the attribution gap. Every query in our benchmark fills a quota-slot from an explicit metric system, rather than because an expert deemed it worth testing—so any score change is attributable to something concrete. Our benchmarks span three orthogonal axes—scenario, complexity, and interaction & risk—with queries generated against quotas by an LLM and verified by experts. Trajectories are scored by multiple LLM judges that must rule unanimously; their disagreements go to human adjudication, whose verdicts refine judging rules. This design also makes benchmarks evolvable: when a benchmark saturates as BlueLM-GUI improves, we do not redesign a harder one—we shift quotas along path complexity, intent dependency, and instruction explicitness, yielding a new benchmark differing only in precisely stated quotas.

Empirically, the real-device flywheel translates into measurable gains. On our MobileGUI-VBench, BlueLM-GUI reaches **87.4**, surpassing the best closed-source model by 5.1 points and setting a new state of the art. On the emulator-based AndroidWorld benchmark ([Rawles et al., 2025](#)), it scores **84.9**—the strongest among open-source and comparably sized models, and competitive with the largest closed-source model despite being orders of magnitude smaller. Together, these results show that grounding Every Sample, Every Rollout, and Every Query in real devices yields strong, robust, and transferable mobile GUI capability. Our contributions are threefold:

- **An all-real-device self-improving flywheel.** We present BlueLM-GUI, a mobile GUI agent whose entire training loop—RL exploration, rollout collection, and evaluation—runs on hundreds of real phones rather than sandboxes or emulators. Every rollout flows back into the data pool, closing a self-improvement loop whose training distribution coincides with deployment.
- **A value-aware trajectory engine that turns failures into supervision.** We design an Error Correction & Derivation Module that triages every trajectory by value: successes become gold-standard supervision, while failed and anomalous trajectories are salvaged into new supervision via step-level correction, retroactive query alignment, and counterfactual derivation.
- **A quota-driven benchmark methodology that evolves with the model.** Our MobileGUI-VBench is defined as a quota distribution over a metric system, enabling precise score attribution; when a benchmark saturates, shifting the quotas yields a new one whose difference is stated exactly.

The remainder of this report is organized as follows. §2 details the data engine, from query generation and real-device collection to trajectory evaluation, failure attribution, and the self-evolution loop. §3 presents the three-stage training recipe. §4 describes the quota-driven benchmark methodology and its evaluation protocol. §5 reports the main results and ablations that validate each principle, and §6 contrasts our approach with concurrent industrial work.

2. Training Data Construction

2.1. Data System Overview

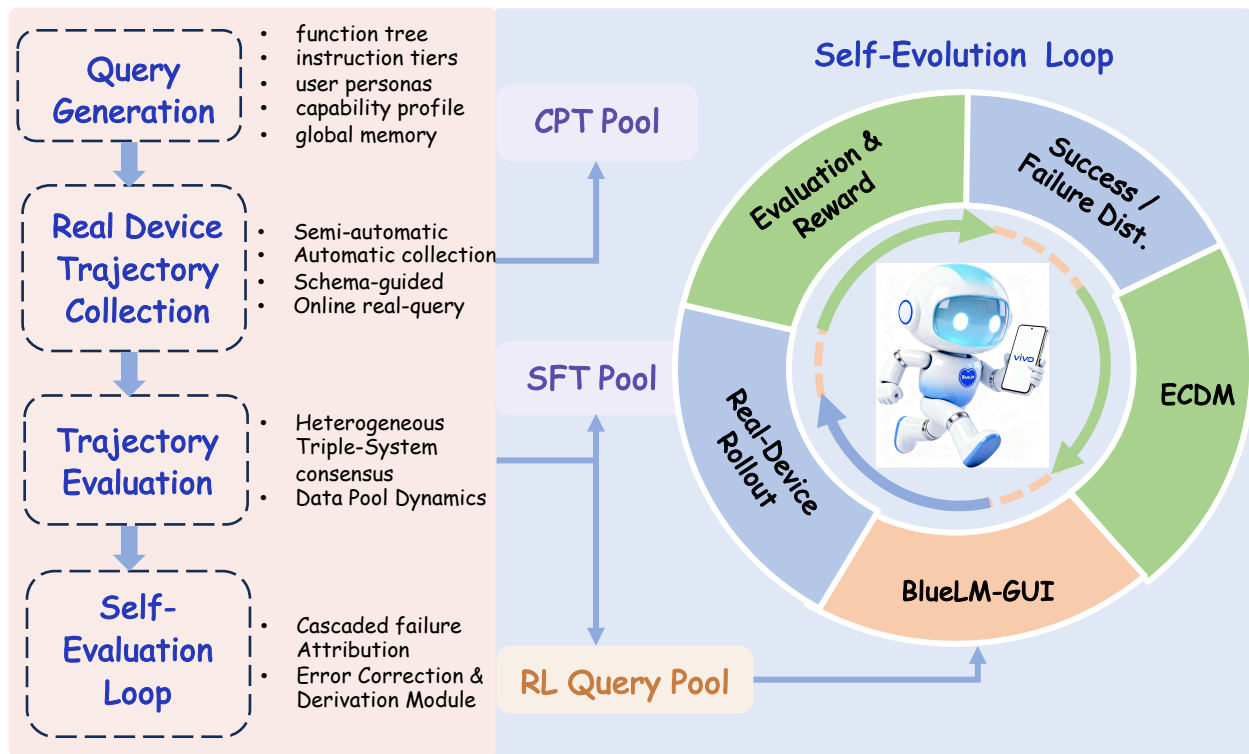


Figure 2 | Overview of the BlueLM-GUI data system.

2.1.1. Design Philosophy

BlueLM-GUI’s data system is built on two complementary foundations: the first defines where the data comes from (the distribution), and the second defines how to make the most of every sample. The first is the all-real-device grounding that the training recipe (§3) elevates into the principle *Every Rollout Is Real*; the second is this section’s own principle, *Every Sample Matters*. Together, they make a real-device-only data pipeline not only feasible, but also capable of maximizing the data value.

Foundation 1: The Real-Device Distribution. All of our GUI trajectories come from real apps on real devices. The prevailing industrial approach pairs real devices with a large number of sandbox/mock environments, which offer resettability, instrumentation, and automated verification. However, they cannot reproduce the long-tail distribution that dominates production failures—risk-control challenges, CAPTCHAs, account states, popup ads, network jitter, and the version fragmentation of super-apps. Collecting on real devices aligns the training distribution with the production distribution and with real-world operating conditions—precisely the conditions under which mobile GUI agents fail in production. This choice is not free—all-real-device collection is time-consuming, hard to reset, and

fragile to verify. This is precisely why Foundation 2 (“Every Sample Matters”) exists: to fully exploit every sample and maximize the value of real-device data.

Foundation 2: Every Sample Matters. Real-device collection is extremely expensive, so every trajectory and every evaluation result is expected to contribute value. Rather than the coarse filtering practice of “keep the clean successes, discard the failures and anomalies,” we extract value from every sample through an **intervene-and-route** philosophy. On the SFT side, trajectories evaluated as successful enter the SFT pool directly as gold-standard data, while trajectories judged as failed or suffering environmental anomalies are not discarded but routed to the **Error Correction & Derivation Module** (ECDM, §2.5.2), which salvages them into new valid supervision. On the RL side, rollout results are likewise routed rather than filtered: boundary cases feed learning and even fully-failed rollouts flow to the ECDM for attribution and repair instead of being discarded (the pool-metabolism rule is detailed in §2.4.3). The core idea is this: failures and anomalies are not waste; they are the scarcest correction and exploration signals.

2.1.2. CPT/SFT Decoupling

The two foundations above define where data comes from and how to make the most of every sample; a further question is how this scarce real-device data should be allocated across training stages. Our answer is to explicitly decouple the **CPT** (Continual Pre-Training) pool from the **SFT** (Supervised Fine-Tuning) pool (see Figure 2)—letting CPT absorb “volume” and SFT guard “quality.”

Asymmetry 1: Production Speed. SFT data is inherently slow to produce: a trajectory must run end-to-end on a real device and pass the long acquisition–evaluation chain before it qualifies, so high-quality SFT data accumulates gradually. CPT data, by contrast, is produced at every collection step: single-step screenshot–action pairs enter the pool immediately after rule-based cleaning, without waiting for the final verdict, so a large volume accumulates quickly. If the two pools shared one intake gate, the fast stream would have to queue behind the slow stream and the model would starve for fresh data. Decoupling lets each stream run at its natural cadence: CPT continuously feeds the model with fresh single-step data at scale, while SFT quietly accumulates high-confidence gold-standard trajectories, and training can proceed with the former instead of stalling on the latter.

Asymmetry 2: Capability Foundation. Before the model can plan GUI tasks, it benefits from first acquiring a set of more general, low-level abilities: reading the screen (OCR, element localization), performing basic interactions, and calling tools (API invocation, parameter construction, result parsing, error handling). These abilities do not depend on long-horizon task success, so the CPT pool deliberately reserves a data ratio for two classes of foundational data: single-step screenshot–action pairs are valid even when the trajectory containing them ultimately fails, and tool-use data (API call pairs, tool schemas, parameter-construction examples) is injected to build a tool-calling prior during pre-training. With these general abilities in place after CPT, SFT can concentrate its scarce supervision on GUI planning itself—and on the decision of “when to use a tool vs. when to operate the interface”—rather than teaching them from scratch. This also reserves the foundational capabilities for the subsequent GUI + API + CLI interaction paradigm.

2.1.3. Dual-Track Pipeline

The pipeline operates as two parallel tracks sharing the same real-device farm. The first track is the SFT data production line, organized as a four-ring main chain: query generation (§2.2), trajectory collection on real devices (§2.3), trajectory-level evaluation via the Heterogeneous Triple-System Consensus (HTSC, §2.4), and data routing with self-healing. Verified trajectories enter the SFT pool, while failures flow to the ECDM for repair. The second track is the RL self-evolution line: queries of

moderate difficulty are rolled out with the latest SFT model on real devices, and the results are routed by the within-group success-failure distribution (the pool-metabolism rule is detailed in §2.4.3). The two tracks thus close a self-healing loop: SFT produces the model, RL probes its capability boundary, failures return through the ECDM as repaired supervision, and the cycle repeats.

The self-healing core spanning both tracks is the **Error Correction & Derivation Module (ECDM)**. It attributes each failed or anomalous trajectory and converts it into new supervision through three repair paths—**step-level correction** (repair a single wrong step via a Teacher model with human fallback), **retroactive query alignment** (RQA, downgrade an over-scoped query to fit the correctly completed prefix), and **counterfactual derivation** (turn recurring failure classes into targeted adversarial queries for the RL pool). The full decision chain—attribution followed by the three paths—is described in §2.5.

2.2. Query Generation

Query generation determines the starting point of the data pipeline and, to a large extent, the diversity of the trajectory samples—which in turn shapes the GUI model’s generalization and robustness. Our design revolves around three dimensions: **functional coverage**, where the function tree systematically maps “what to do” to reduce blind spots; **instruction diversity**, where instruction tiers and user personas keep “how it is said” close to natural instruction distributions; and **task complexity**, where capability profiles together with Global Environment Memory elevate single-point tasks into complex tasks with genuine reasoning content.

2.2.1. Function Tree

Real commercial apps do not expose functionality as a flat list, but as a complex hierarchy of home-page entries, channel pages, search-result pages, detail pages, and action pages. We maintain a **function tree** for each client application, built through a semi-automatic pipeline:

- **Automatic exploration**: an exploration agent automatically traverses the app, screenshotting each page layer and extracting its layout schema.
- **Semantic extraction**: a strong multimodal model (VLM) identifies UI interaction components, copy text, and latent functionality, automatically generating a JSON tree structure.
- **Human pruning**: humans prune conflicting and redundant nodes.
- **Reuse-relation tagging**: for the same functionality reachable via multiple paths (e.g., entering the same page via “Me → Settings → Offline Packs” and via “Home → top-right corner → Offline Packs”), the tree marks a reuse relation to avoid duplicate expansion.

Figure 3 shows an example function tree for a translator app. The tree then serves as the unified index across the whole pipeline: query generation expands over leaf nodes, trajectory collection navigates toward them as targets, and coverage measurement computes the fraction of leaf nodes already exercised—the **function-tree coverage** metric, which turns “full functional coverage” into a measurable quantity.

2.2.2. Instruction Tiers

To train the agent’s adaptability to users with different levels of expression clarity, every leaf node on the tree is mapped to queries at three tiers. The higher the tier, the closer to real colloquial speech, and the more it tests the model’s intent understanding and generalization.

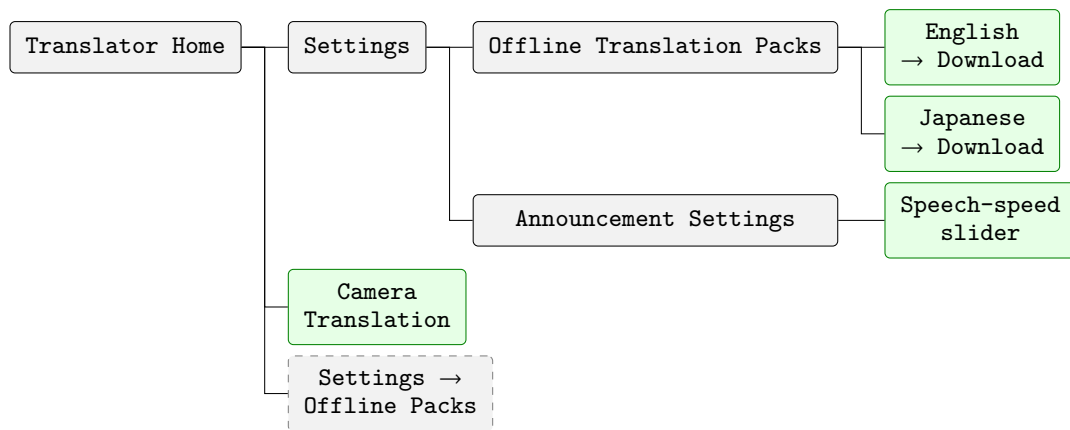


Figure 3 | An example function tree of a translator app. Green nodes are leaf targets; the dashed node marks a reuse relation.

- **L1 step-by-step instruction:** the user knows the exact path and guides with explicit steps.
 - *Plain narration*: “Tap Settings on the home page, tap Offline Translation Packs, and choose Download on the English card.”
 - *Colloquial steps* (same path, colloquial wording): “First tap Settings on the home page, then pick that Offline Translation Packs thing inside, and finally download the English one.”
- **L2 explicit goal-oriented instruction:** the user’s goal is crystal clear, but no path is given. By semantic emphasis, we subdivide this tier into six subclasses for finer coverage:
 - *Function statement*: “Go download an English offline translation pack.”
 - *Purpose statement* (implies the functional purpose): “I’m about to board a plane and won’t have internet—grab an English offline pack just in case.”
 - *Dependent operation* (long-chain before/after dependency): “Download an English offline pack, and once it’s done switch to real-time voice translation.”
 - *Parallel operation* (multi-tasking): “Download an English offline pack, and while you’re at it make the font on the translation screen a bit bigger.”
 - *Conditional operation* (branching): “Check whether there’s a Burmese offline translation pack in the app, and if there is, download it right away.”
 - *State/spatiotemporal-constraint operation*: “I’m flying to Tokyo on a business trip the day after tomorrow—get me a usable offline pack for the local language.”
- **L3 ambiguous / colloquial instruction:** simulates highly irregular colloquial expressions observed at high frequency.
 - *Incomplete instruction* (the goal is only hinted at): “Get me that one that works without internet.”
 - *Intent inference* (no functional words at all): “I’m heading abroad soon with no internet—how is this thing supposed to look up words?”
 - *Colloquial noise / self-correction* (the user changes the goal mid-sentence): “Hey, that English one, download the local pack. . . wait no, I want the Japanese one!”

2.2.3. User Personas

Even within the same instruction tier, different user groups exhibit distinctly different language habits. If the function tree determines what functionality is involved and the instruction tiers determine how the user says it, then the user personas determine who is saying it. We predefine eight standard user

personas; during instruction generation, the LLM role-plays a specific persona and applies a style filter over the query’s tone, wording, and habits for the same functionality point, suppressing the “machine accent” of synthesized data:

Persona	Style	Example Query
Average user / novice	Plain speech	“Download the English offline pack—worried it won’t work without internet.”
College student / Gen Z	Internet slang	“Quick, grab the offline translation—boarding soon and there’s no signal!”
Programmer	Terse and direct	“English offline pack. Download.”
Business professional	Formal and precise	“Please download the English offline translation pack for use during my flight later.”
Senior citizen	Rambling / anxious	“Oh dear, the internet’s about to cut out—will that translation thing still work? Hurry and save one for me.”
Delivery rider	Urgent and clipped	“English one, the offline kind. Quick.”
Travel blogger	Enthusiastic and lively	“Help! Flying to Thailand soon with no internet—save me a translation pack, quick!”
Stay-at-home mom	Gentle, multitasking	“Once the baby falls asleep, download the offline translation for me, and while you’re at it check some baby-food recipes.”

Table 1 | The eight standard user personas and their example queries.

2.2.4. Capability Profile

The function tree determines which business domain is involved, and L1–L3 determine how the user says it (the expression axis). The **capability profile** adds the orthogonal complexity axis: it determines how hard the task is. Instruction tiers and user personas diversify expression without changing the intrinsic difficulty of the task, whereas the capability profile stacks reasoning and operational load onto the task, elevating a simple “single-point functionality task” into a “multi-step, cross-app complex task demanding rigorous logical reasoning.”

To operationalize the complexity axis, we define six core capability dimensions: the first three raise the decision difficulty within a single point, and the last three stretch the task structure across pages and apps. When generating tasks, the LLM is guided to project tasks onto these dimensions:

- **Multi-Constraint Filtering:** the model must identify and activate multiple filter conditions in the interface (e.g., in Meituan: “find a Japanese restaurant with per-person spend under 100 CNY, a rating above 4.5, and delivery support”).
- **Numerical Reasoning & Calculation:** the model must make mathematical decisions such as price comparison, currency conversion, and coupon stacking (e.g., “select the first three items in the cart and figure out which combination with the spend-and-save coupon is the best deal”).
- **Conditional Logic:** the model chooses different execution paths based on runtime environment state (e.g., “check whether this delivery order has free delivery—if yes, place the order; if not, switch to another restaurant”).
- **Cross-Page State Tracking:** the model must maintain short-term memory across long operation chains, using information extracted on Page A as input on Page C (e.g., “remember the model number of that Sony headphone we just saw—we’ll need it for price comparison later”).

- **Looping & Iterative Execution:** the model must repeat actions over a series of similar components (e.g., “download every PDF posted in the group chat today and save them all locally”).
- **Information Synthesis:** the model must gather scattered information across pages and apps and organize it into output (e.g., “check Ctrip and Qunar separately for the cheapest flight to Beijing tomorrow, and send it to Zhang San on WeChat”).

The structural dimensions—cross-page state tracking, looping, and information synthesis—cannot be exercised within a single point or a single app, and even the decision dimensions compound in difficulty when combined across functionality points. The capability profile therefore pushes tasks toward cross-app, multi-functionality combinations—and this, in turn, imposes requirements on the physical reachability of synthesized instructions, which is exactly the problem addressed by Global Environment Memory in the next section.

2.2.5. Global Environment Memory

However, an LLM generating such cross-app tasks has no direct access to the real state of the target device. Synthesized cross-app instructions are therefore often physically unreachable: an LLM might fabricate “read the tracking number in Xiaohongshu and paste it into Cainiao to query the parcel,” yet Xiaohongshu’s page may block screenshots due to risk control, or Cainiao may not be logged in on the current device at all. To bridge this gap, we maintain a **Global Environment Memory (GEM)**: a topological graph database that structures reachability knowledge on real devices, so that every step of a synthesized instruction has a physical foothold. GEM serves three roles in the pipeline: it is built from and updated by real-device exploration, it grounds cross-app query generation, and it receives failure feedback from the ECDM to keep itself fresh.

GEM Construction and Update Flow. GEM combines two complementary knowledge sources. The first is the page-navigation graph obtained from real-device exploration, built in four steps:

- **Active environment exploration:** when the real-device farm is idle, exploration agents automatically perform non-destructive walkthroughs on real devices, recording page screenshots and component trees (Layout XML).
- **Perceptual deduplication:** pHash (perceptual hashing) + layout-similarity algorithms filter duplicate pages, keeping only page skeletons with unique functional states.
- **Navigation-graph construction:** build an adjacency navigation graph between pages. Edges come in two types:
 - *Intra-app navigation edges:* normal page navigation and interaction transitions.
 - *Cross-app navigation edges:* paths that switch via system-level media such as the Home key, the recents panel, the share sheet, or the clipboard.
- **Functionality and data-flow compatibility extraction:** a strong VLM analyzes the “input/output compatibility” of each page (e.g., the output of a “Taobao product-detail page” is `Text:product title`; the input of a “JD.com search page” is `Text:search keyword`; the two are semantically type-compatible).

The second source is device- and system-level state knowledge, available to us as a device manufacturer: functionality differences across device models and OS versions, exact paths to system settings, and the capability boundaries of pre-installed apps. This knowledge is injected into GEM as precise priors on functionality existence and paths, complementing the verified navigation reachability from real-device exploration. Together, the two sources ensure that synthesized queries are tailored to the target device’s real environment, rather than exposing path mismatches only at collection time.

Cross-App Task Generation Flow. With GEM, generating complex cross-app instructions such as “price comparison” and “relay” tasks shifts from ungrounded guessing to grounded generation after graph-path retrieval:

- **Choose a start point:** randomly sample a start point from the function tree (e.g., search head-phones on Taobao).
- **Path exploration:** query the GEM graph database for a cross-app endpoint whose input/output is compatible with the start point (e.g., JD.com search).
- **Path-existence check:** verify that the physical path “Taobao → launcher → JD.com” exists and is traversable.
- **Generate grounded query:** the VLM combines the real page contexts of both apps to generate a reachable query.

Closed-Loop Interaction with the ECDM. When a real device hits an `Env_ERROR` while executing an instruction, the failure signal flows back to GEM: the broken path is marked “temporarily unreachable” and instruction generation over that path is halted, while query generation reroutes to a reachable alternative path in the updated graph. This keeps GEM synchronized with the ever-changing real-device environment.

2.2.6. Query Intake Processing

Queries reach the collection stage through two channels—newly synthesized and externally imported—and each channel receives a different treatment beforehand: synthesized queries pass a quality gate at generation time, while external stock queries are post-hoc tagged to fit the same label system.

Quality Gate for Newly Synthesized Queries.

- **Gate 1: Three-axis evaluation by an LLM judge.** Newly generated queries must pass an independent LLM judge on three dimensions:
 - *Realism*: whether the query conforms to genuine human expression habits (hard filter).
 - *Completeness*: whether it is physically executable given the functionality currently present in mainstream apps (hard filter; failures are discarded outright).
 - *Complexity*: evaluates operation-step count and reasoning depth; not used as a hard filter, but stored as a metadata label for fine-grained mixing ratios and RL-pool candidate screening (§2.4.3).
- **Gate 2: Natural-language polishing (denoising & polishing).** A lightweight rewriting model converts text with a heavy “LLM-prompt accent” into idiomatic expression with natural pauses, colloquial noise, and abbreviations (echoing the persona styles of §2.2.3).
- **Gate 3: Back-tagging to function-tree nodes.** An algorithm automatically anchors each approved query back to the corresponding leaf or subtree node of the function tree (§2.2.1), enabling real-time monitoring of function-tree coverage and node-weighted training mixtures.

Post-hoc Four-Dimensional Tagging of External Stock Queries. For stock queries from curated external sources, business BadCases, or open-source datasets—which were not produced by this pipeline and carry no production labels—we run a round of four-dimensional post-hoc tagging with an LLM, so that they become comparable and mixable with internally produced queries under one unified label system:

- **Step depth:** the estimated minimum number of operation steps required.

- **Ambiguity grade:** aligned and classified into L1 / L2 / L3.
- **Risk grade:** flags whether the query involves high-risk actions such as financial payment, privacy exposure, or sensitive account-information modification.
- **Pre-requisites:** whether the query depends on specific physical preconditions (e.g., “bank card bound,” “account logged in”), so the real-device farm can automatically preset the environment before execution.

2.3. All-Real-Device Trajectory Collection

After query generation, the next step is to turn an abstract query into a high-quality real-device trajectory carrying a fine-grained reasoning chain. We built an all-real-device farm hosting three parallel collection lines plus one auxiliary external query channel (§2.3.1); every trajectory carry structured reasoning chains (§2.3.6) and then pass horizontal enrichment (§2.3.7) before entering the pools.

2.3.1. Trajectory Collection Methods Overview

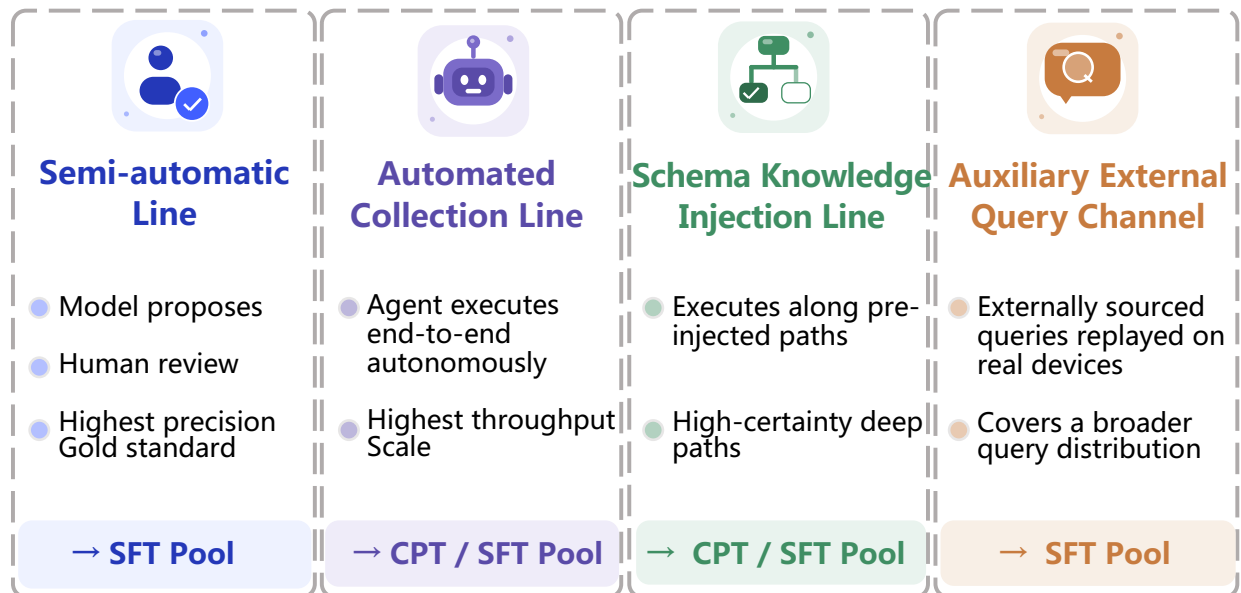


Figure 4 | The four collection channels of the BlueLM-GUI data system: three parallel collection lines (semi-automatic, automatic, schema-guided) plus one auxiliary external query channel.

Different tasks demand different trade-offs between precision, cost, and throughput, so we run four collection channels on the all-real-device farm—three parallel lines plus one auxiliary external query channel—as illustrated in Figure 4:

- **Semi-automatic line** (model proposes, human verifies each step): highest precision, for gold-standard data—verified trajectories go straight to the SFT pool (§2.3.2).
- **Automatic line** (agent policy runs end-to-end, unsupervised): highest throughput, for scale—raw trajectories are routed by HTSC evaluation (§2.3.3).
- **Schema-guided line** (model follows a pre-injected operation path): for high-determinism deep flows (system settings, vertical apps) where blind exploration collapses—trajectories are routed by HTSC evaluation (§2.3.4).

- **Auxiliary external query channel** (externally sourced queries replayed on real devices): for covering a broader query distribution—replayed trajectories are routed by HTSC evaluation (§2.3.5).

2.3.2. Semi-Automatic Collection

Semi-automatic collection is our primary gold-standard production line. It adopts a “model proposes, human verifies (HITL)” collaboration mechanism: the model contributes fast reasoning, and the human ensures step-level correctness. Because a human performs “surgical” verification at every step, this line keeps hallucinated trajectories out of the data and preserves the device time invested in the preceding successful steps.

- **Dual-mode recommendation:** the annotation client pulls the current real-device screenshot and queries the LLM, which outputs a structured Chain-of-Thought (CoT) and a predicted action.
- **Micro-adjustment verification:** if the recommended action is accurate, the verifier confirms with one click; if it deviates (e.g., the tap position is off by tens of pixels, or a “slider drag” is misjudged as a “tap”), the verifier directly drags the tap box to fine-tune it, modifies the action type, or rewrites the text on the screen image to correct it.
- **Execution and iteration:** the system executes the verified action on the real device, captures the new screen, updates the state, and proceeds to the next step until the task completes.

2.3.3. Automatic Collection

The automatic line trades per-step verification for throughput: the agent model drives tasks end to end, so collection volume is far higher than the semi-automatic line, at the cost of step-level correctness, which physical-level safeguards and post-hoc replay correction compensate for.

- **Closed-loop execution flow:** the scheduling hub dispatches a query, and the model autonomously runs the observe–decide–act loop, judging for itself whether the task is complete or ending on a stop condition (timeout, maximum steps exhausted, or consecutive repeated actions).
- **Physical-level self-cleaning and safety isolation:**
 - **Physical state reset:** before each task, the control system force-stops all background processes (`am force-stop`) and clears the target app’s data (`pm clear`) via low-level ADB commands, so every launch starts from a clean initial state.
 - **Physical anti-risk-control disguise:** devices are bound to dedicated physical SIM cards on independent cellular networks; instead of “teleport”-style coordinate taps with no transition trajectory, the system injects touch-trajectory signals with realistic physical acceleration, evading commercial apps’ device risk-control engines.
- **Post-hoc triage and repair routing:** without per-step verification, automatic trajectories may mix in cases of mid-task disorientation, misjudged completion, and interruptions by popups or ads. These trajectories are flagged and funneled into the HTSC evaluation routing of §2.4.

2.3.4. Schema-Guided Collection

On-device settings and deep vertical apps (e.g., banking apps, developer options) have deterministic yet complex operation paths, where slight visual or copywriting differences can collapse a blind exploration’s success rate. We therefore developed the **Schema-Guided Rollout** line: the known operation path of a functionality point is injected into the system prompt before execution, so the model follows the path instead of exploring. It supports two modes:

- **Tap-operation mode:** inject the complete operation path (e.g., “Settings → WLAN → Advanced Settings → MAC Address”); the model taps through it step by step, skipping trial-and-error exploration; when it encounters an irreversible toggle-type operation or a system confirmation dialog, it emits a clarification to the user and proceeds only after confirmation.
- **Search-operation mode:** inject a search term plus a functional-location description (e.g., “search ‘Bluetooth’ in Settings, tap the second result”); the model locates the target page through the search box instead of drilling down layer by layer.

2.3.5. Auxiliary External Query Channel

Beyond the three synthesized-query lines, we operate an additional channel that brings in externally sourced queries, complementing the synthesized-query distribution:

- **Query pull:** pull queries for GUI-skill tasks from curated external channels.
- **Cleaning and intent classification:** all queries undergo multi-stage anonymization and privacy audit, followed by LLM-based cleaning and multi-level intent classification, removing invalid, sensitive, and non-executable samples.
- **Real-device automatic replay:** replay these queries on real devices via automatic execution tools, producing structured trajectories with step-by-step `cot` / `action`.
- **Result routing:** replayed trajectories enter the HTSC evaluation of §2.4 just like synthesized-line trajectories—those that pass are corrected and enter the training pool, while failures go to the ECDM for attribution and repair.

2.3.6. Structured CoT

GUI task execution involves long-horizon sequential decisions; a model trained purely on action sequences cannot generalize its reasoning to previously unseen pages. We therefore require every collected trajectory to carry a structured reasoning chain before each action, embedding both “what to do (What)” and “why (Why)” directly in the data. The schema is written identically into the system prompt at inference time, making it the data foundation for process supervision in the subsequent SFT phase.

Five-Tag Structured CoT. Each action produced during collection is preceded by the five tags (Table 2):

Tag	Meaning	Example
[Observation]	Description of the current page state	“Currently on the Taobao search-results page; a search box at the top, a product list below”
[Reflection]	Review of and correction to the previous step (triggered only when the observation conflicts with the previous step’s plan)	“Tapped the filter button last step, but the page didn’t change—the tap position may have been off”
[Plan]	Next-step plan	“Tap the search box and type ‘Sony XM5’ to search”
[Decision]	The concrete action	“Tap the search box at coordinates (540, 200)”
[Memory]	Information to remember across steps	“Remember: the product currently being compared is the Sony XM5, priced at 1,999 CNY”

Table 2 | The five tags of the structured CoT.

Bi-Directional Consistency Constraint. To prevent “post-hoc rationalization” (generating the action first, then assembling plausible-looking reasoning around it), a bi-directional verification gate runs before trajectories are replayed into the pool:

- **Forward derivability:** given [Observation] + [Plan], the [Decision] must be the uniquely reasonable action—different actions must not all “make sense.”
- **Label consistency:** the action in [Decision] must exactly match the action executed in the trajectory; the page state in [Observation] must match the screenshot; the [Memory] items must actually be referenced in later steps. Any violation triggers regeneration until all pass.

2.3.7. Horizontal Enrichment

Raw trajectories collected from each line pass through a unified horizontal enrichment pipeline, which reorganizes and refines existing trajectories or intervenes during collection without creating new tasks, so that the model gains stronger generalization and robustness. The pipeline comprises three classes of processing:

- **Step-level context disambiguation:** different apps can have extremely similar visual structures (e.g., the near-identical “shopping cart” pages of various supermarket apps). In each step’s trajectory metadata, we explicitly write the current system-level state obtained via ADB at collection time—the foreground app package name (e.g., `com.taobao.taobao`) and the Activity stack—giving the VLM an unambiguous physical identifier for multi-dimensional reasoning in cross-app scenarios and reducing the drift of purely visual reasoning.
- **Anomalous state injection:** to train an agent that keeps functioning under harsh conditions, we inject two classes of unexpected disturbances during real-device collection—*system interruptions* (mid-operation push notifications, low-battery alerts, or pre-set permission dialogs such as “Allow location access”) and *network fluctuations* (artificially induced latency or disconnection that forces a “Network unavailable, please retry” placeholder). Capturing the model’s successful self-recovery paths under these disturbances (e.g., tapping “Allow” on a dialog, or tapping “Refresh”) builds its defensive playbook. For anomalous states that cannot be injected—login-session expiry, CAPTCHAs, payment verification—we collected roughly 5,000 targeted samples covering 14 classes: the model learns to stop and hand back control where human intervention is required (e.g., CAPTCHAs), and to close or bypass safely skippable pages (ads, non-critical popups).
- **Repetition-to-reflection:** real-device rollout trajectories often contain repeated segments where “a single action or a small group of actions is executed multiple times in a row” (from local failure modes or page lag / network jitter). Used directly as training labels, these would teach the model to “repeat when stuck.” We remove the repeated steps from the training labels but keep them in the trajectory history (as evidence of “possible stalling”); when a later step breaks out of the stall (going back / switching entry / replanning), that corrective step remains a label, with the earlier failed attempts still in its context—turning a harmful label pattern into a reflection- and error-recovery-oriented supervision signal that works in concert with the [Reflection] field of the structured CoT (§2.3.6).

2.4. Trajectory Evaluation and Data Routing

In an industrial-grade GUI-agent data pipeline, trajectory-quality judgment and efficient data routing form the hub that largely determines how far model capability can evolve. We therefore designed a **two-phase routing architecture** paired with **Heterogeneous Triple-System Consensus (HTSC) evaluation**, ensuring that every trajectory—whether collected on real devices or rolled out

autonomously—is precisely routed and fully utilized, putting the “Every Sample Matters” philosophy into practice.

2.4.1. Two-Phase Evolution of Data Routing

A query’s routing difficulty can only be measured against an existing policy (e.g., “moderate difficulty” defined by success rate), yet no such policy exists at cold start. We therefore split the routing mechanism into two phases according to whether a current policy exists. The HTSC verdict criterion itself is phase-invariant—it always judges whether the trajectory physically completed the task; what evolves across phases is the difficulty yardstick for routing, which applies to the RL query pool:

- **Phase 0 · Bootstrap:** no usable GUI policy model yet. Trajectory routing relies on policy-independent objective assertions only: HTSC judges correctness, and the three data pools fill according to their fixed rules—the CPT pool accumulates single-step screenshot–action and tool-use corpus via its inspection-free bypass (not routed by HTSC; §2.4.3), while the SFT pool accumulates gold-standard corpus from HTSC-verified and human-adjudicated trajectories, and the RL query pool is pre-filled by objective difficulty features (its phase-by-phase metabolism is detailed in §2.4.3). The core goal of this phase is to train the initial policy model $M^{(0)}$.
- **Phase 1 · Self-Evolution:** the current policy model $M^{(t)}$ is now in service. The difficulty yardstick upgrades to policy-dependent: the actual rollout success/failure distribution (success rate) of $M^{(t)}$ over the query pool serves as the yardstick, selecting samples with the highest learning signal at the current capability boundary to drive the coordinated iteration of RL and SFT (see §2.4.3).

2.4.2. Heterogeneous Triple-System Consensus

In the overall flow diagram (Figure 2), “trajectory evaluation” is the pivotal checkpoint. A single evaluator—however capable—suffers from multimodal hallucination and single-perspective blind spots, so its verdict cannot be trusted as the sole gatekeeper of training-pool purity. We therefore built the Heterogeneous Triple-System Consensus (HTSC) mechanism: three evaluators with heterogeneous technology stacks cross-validate the same trajectory under a shared safety veto, a consensus decision maker aggregates their verdicts into one of four outcomes, and each outcome maps to a fixed routing destination. This section describes these stages and the downstream routing in order.

Stage 1 · Safety Veto (pre-screen). Safety is enforced by a shared safety checklist that all three systems execute in parallel with their correctness evaluation—covering scenarios such as direct payment, sensitive permissions, privacy-data access, and irreversible dangerous operations. A safety violation detected by any system triggers a one-vote veto: the trajectory bypasses the consensus vote entirely, is judged `False` outright, and is routed to the ECDM—safety red lines do not participate in majority voting, and there is no channel through which “most systems see no problem” can override.

Stage 2 · Three Heterogeneous Evaluators. The three systems divide the work along different evaluation dimensions, cross-validating the same trajectory from three levels—terminal state, key milestones, and overall path—and reduce the risk of common-mode failure through heterogeneity of the underlying technology stacks:

- **System A (terminal-state physical verification):** focuses on factual verification via system-level data—reading OS-level APIs, database logs, network traffic, and filesystem state—to make hard-metric assertions about the task’s terminal state (e.g., “has the file been downloaded,” “has the setting been enabled”). Highest decision certainty, but only covers tasks whose terminal state is readable from the device side.

- **System B (milestone-decomposition verification):** focuses on two-granularity “milestone + global” analysis—first decomposing the task into an ordered subgoal checklist, checking along the trajectory whether each milestone is reached, then making a global judgment on the overall result. It answers “were all the key nodes traversed correctly.”
- **System C (overall-path plausibility verification):** focuses on semantic-level evaluation of the overall task and planned path—consuming the long-horizon Action-Observation history and judging whether the trajectory’s progression logic is coherent, and whether there is meaningless detouring or pseudo-completion (appearing to progress while not actually reaching the goal). It answers “is this path reasonable.”

Stage 3 · Consensus Voting. The consensus decision maker aggregates the three systems’ verdicts into one of four outcomes—True (task completed), False (model execution failure), Env_ERROR (failure attributable to environmental factors), or Undetermined (no consensus). Each system votes from within the label space {True, False, Env_ERROR, abstain} (System A abstains when it cannot read the terminal state; Env_ERROR is flagged upon detecting popup interception, network anomalies, missing permissions, or app crashes). Voting proceeds by the following priority:

- **System A can read the terminal state:** A’s physical assertion dominates the decision as a high-confidence fact. An A assertion of failure yields False (or Env_ERROR if A identifies an environmental cause) regardless of B and C; an A assertion of success yields True unless B and C unanimously object, in which case the outcome is Undetermined.
- **System A cannot read the terminal state (abstains):** Systems B and C vote—unanimous pass yields True, unanimous failure yields False, and disagreement yields Undetermined.

Routing of the Four Outcomes. Each consensus outcome maps to a fixed destination—the verdict decides where the trajectory goes, and no trajectory is ever discarded:

- **True → SFT pool:** a high-quality successful trajectory, injected directly into the SFT pool.
- **False → ECDM:** confirmed algorithmic execution failure, sent to the correction module for step-level repair or retroactive alignment (§2.5.2), after which it becomes high-quality SFT data.
- **Env_ERROR → ECDM:** an objective failure caused by environmental factors—not counted as a model error. It triggers the environment self-healing mechanism and re-initiates collection; meanwhile, the correction module can synthetically derive “close the popup and resume the task” repair steps into the trajectory, recovering usable supervision from these samples.
- **Undetermined → human adjudication:** detailed below.

Human Adjudication of Undetermined Samples. An Undetermined sample should neither rashly enter the gold-standard pool and pollute SFT, nor rashly be sent to correction as a failure, so it is routed to human adjudication: human judges correct → SFT pool; human judges wrong → ECDM (§2.5.2); human judges environment issue → the Env_ERROR branch with environment self-healing and re-collection. Beyond settling individual samples, this loop compounds: disagreement samples are the evaluation system’s most valuable “stress test,” concentrating exposure of single-modality judges’ blind spots, and the human verdicts are simultaneously used to retroactively calibrate the thresholds and prompts of the evaluation systems, so HTSC’s consensus rate keeps rising across iterations. This routing design puts Every Sample Matters into practice: a verdict disagreement is itself a supervision signal, not noise to be smoothed away by majority vote.

2.4.3. Data Pool Dynamics

Evaluation (above) together with attribution and ECDM repair (§2.5) yield finished data; this section answers its final destination: the three data pools. What separates them is less *what* they store than whether a sample’s value persists or shifts as the model $M^{(t)}$ evolves—two pools accumulate monotonically, while the RL pool must metabolize.

- **CPT pool—static accumulation, append-only.** The single-step “pixel–element–action” correspondence is independent of the trajectory’s final success (§2.1.2), and its correctness does not change with the policy either: a valid single-step sample is equally valid for $M^{(0)}$ and for $M^{(t)}$. The pool therefore only appends, never retires—massive single-step data flows in inspection-free and accumulates continuously.
- **SFT pool—static accumulation, purity first.** It receives only three classes of high-confidence trajectories (HTSC-verified True, human-adjudicated correct, and ECDM-repaired qualified; §2.4.2, §2.5.2). Correctness, once confirmed, holds indefinitely, so this pool likewise grows monotonically; it differs from the CPT pool only in intake-gate strictness, not in temporal behavior.
- **RL query pool—dynamic metabolism, difficulty moves with the model.** The one exception: a query’s RL value depends not on the query itself, but on whether the current model can solve it. As the model strengthens, yesterday’s boundary case becomes today’s easy win, and the pool must metabolize accordingly—otherwise RL compute is wasted on already-mastered queries.

The RL pool’s metabolism therefore proceeds in two phases (motivation in §2.4.1):

- **Phase 0 (Bootstrap):** no policy exists to serve as a difficulty yardstick, so policy-independent objective features stand in—instruction tier L2/L3, large step depth, cross-app linkage, multi-functionality combinations. Naturally difficult queries are pre-screened and banked in advance, to be activated once $M^{(0)}$ is trained.
- **Phase 1 (Self-Evolution):** after each round of RL training, $M^{(t)}$ performs multiple rollouts over the pool’s queries, and the within-group success/failure distribution drives a three-way metabolism at the query level: All True → retirement (mastered, no learning signal remains); mixed outcomes → retention (at the capability boundary, highest gradient information gain); All False → the query leaves the RL pool and its failed trajectories flow to the ECDM (beyond the boundary; repaired into SFT corpus to help broaden it).

Intake into the RL pool comes from multiple channels: cold-start objective-feature screening, data-pipeline routing, business-side supplementation, and targeted supplementation of BadCases exposed online / in evaluation. Under Phase 1, all channels pass a uniform gate—“sort by the current policy’s success rate and remove all-true / all-false”—ensuring every entering query falls in the medium-success-rate zone of the current policy.

It is precisely this structure—two static pools as the foundation, one dynamic pool probing the boundary—that lets the data system both accumulate (correctness endures) and keep pace (difficulty moves with the model): the reservoir on which the self-evolution loop of the next section runs.

2.5. The Self-Evolution Loop

The preceding sections (§2.2–§2.4) are “one-shot” data production: queries are generated, trajectories collected, evaluated, and routed into pools—producing a batch of static corpus. But a GUI agent’s

capability ceiling depends on whether it can continuously update its data as it improves. The self-evolution loop introduced in this section is the engine of “Phase 1”: starting from the initial model $M^{(0)}$ trained during Phase-0 bootstrap, it drives the model and the corpus to co-evolve across rounds.

Limitations of Static Corpus. GUI execution is sequential decision-making: a single mis-tap at step i changes the state distribution at step $i + 1$, and the error cascades through all subsequent steps. This gives static corpus two structural limitations: supervision is mostly **off-policy** (the state distribution of annotated demonstrations mismatches the distribution the current model actually visits) and **success-biased** (“correct actions in correct states” are over-represented). The model thus only learns “how to continue on the right track,” while supervision is thin on “how to recognize a wrong state” and “how to recover”—the main bottleneck of reflection ability and complex-task performance.

The loop addresses the two limitations to different degrees. **Success bias** is covered directly: rollout failures are no longer discarded but converted by the ECDM into three classes of supervision—single-step correction labels within erroneous states, short-chain demonstrations anchored on correct prefixes (RQA), and adversarial tasks targeting weak capabilities—so failure and recovery signals continuously enter the training distribution. **The off-policy gap** is covered progressively: ECDM-repaired labels remain offline rewrites, but each round starts from rollouts of the current model $M^{(t)}$, so the repaired supervision is anchored on the states the current policy actually visits, and the train-visit distribution gap narrows across rounds; the RL line further provides strictly on-policy gradients, complementing the SFT line.

Single-Loop Self-Evolution Dynamics. Once $M^{(0)}$ is trained in Phase 0, the loop runs at full capacity, each iteration executing four serial steps:

Model rollout → evaluation and cascaded attribution → error correction & synthetic derivation (ECDM) → retrain and re-rollout

The loop is error-driven: rollout blind spots (All False and Env_ERROR) become ECDM input, and the repaired corpus upgrades the model to $M^{(t+1)}$, which attacks harder boundary cases—model and corpus co-evolve across rounds. Among the three destinations of the attribution step—environment self-healing, evaluation-system revision, and ECDM repair—it is the third that closes this loop (the right half of Figure 2). As iterations proceed, simple errors disappear from rollouts and correction data tracks the model’s capability ceiling. We now detail the loop’s two core steps in turn: cascaded attribution (§2.5.1), which decides where each failure goes, and the ECDM (§2.5.2), which turns it into data.

2.5.1. Cascaded Failure Attribution

When a trajectory flows into the False or Env_ERROR branch, the first step of the ECDM is cascaded attribution: failure causes are determined level by level, in a fixed order—engineering side first, algorithm side last—and the conclusion at each level directly decides the trajectory’s subsequent processing path within the ECDM. The cascade is necessary because environmental interference and model deficiencies are often isomorphic in their trajectory-level appearance: a mid-task interruption caused by a network disconnection and one caused by lost state tracking are nearly indistinguishable in the Action-Observation history. Without first excluding objective environmental factors, environment noise would be misattributed to capability gaps, and the ECDM would then generate repair directions aimed at the wrong target.

The attribution engine executes three levels of diagnosis in the following order:

- **Level 1: environment-failure check (objective, device-side facts).** Examine whether the trajectory contains observable environmental interference—system-popup interception, network anomalies, missing permissions, app crashes. If the failure is directly attributable to such factors, it is labeled `Env_ERROR`: the trajectory is not counted as a model error, and the structured interference features (popup type, occurrence timing, network state) are extracted and fed back to the environment-maintenance module, triggering automated self-healing of the real-device environment and re-collection.
- **Level 2: task- and verifier-defect check (data-production-side).** With environment factors excluded, examine whether the failure originates from the data-production side rather than the model, separating two sub-cases by owner: a *task-specification defect*—an ambiguous, self-contradictory, or not-verifiable-as-written instruction—is a query-generation fault, so the offending task description is returned to query generation for revision; a *verification-rule defect*—a rule that failed a correctly completed trajectory—is an evaluation fault, so the rule is corrected and the mis-judged trajectory re-enters the evaluation queue. Neither is charged to the model. A well-posed task that merely exceeds the *current* model’s capability is not a Level-2 defect—it is handled at Level 3 by RQA.
- **Level 3: model-failure attribution (algorithm side).** Only after the first two levels are excluded is the failure attributed to the model itself, and further subdivided into structured causes: *perception drift* (the VLM mis-identifies or misses UI elements, including localization offsets and element confusion), *state-tracking errors* (losing context across page transitions, causing looping behavior), and *constraint violations* (failing to follow specific restrictions in the query). The attribution conclusion maps directly onto the ECDM’s three processing paths: occasional single-step slips go to step-level correction, well-posed tasks beyond the current model’s capability go to Retroactive Query Alignment (RQA), and recurring error classes go to counterfactual query derivation (§2.5.2).

This cascade turns “evaluation signals” into “executable optimization targets”: every level’s conclusion has a definite downstream destination—environment features feed environment self-healing, task-specification defects feed query-generation revision and verification-rule defects feed evaluation revision, and model-failure causes feed ECDM repair and the next round of directed data production—so every diagnostic signal has an owner.

2.5.2. Error Correction & Derivation Module

The Error Correction & Derivation Module (ECDM) receives all failed trajectories produced by each round of rollout and evaluation routing. Its task is to turn failure into data. All processing is fully offline pure data engineering, consuming no real-device collection resources. Upon entry, each failed trajectory is dispatched to one of three processing paths by directly consuming the conclusion of the cascaded attribution (§2.5.1)—the attribution engine has already determined “at which level the failure occurred,” and the ECDM does not re-judge:

- **Occasional single-step slip** (prefix correct; only individual steps went wrong, and the correct action at those steps can be determined offline) → step-level correction → repair the trajectory → SFT pool.
- **Task over-scoped** (trajectory prefix correct, but the original query exceeds capability or contains unreachable subgoals) → Retroactive Query Alignment (RQA) → downgrade the query to match the prefix → SFT pool.
- **Capability gap** (the same error class recurs across multiple trajectories, pointing to a capability deficiency) → counterfactual derivation → batch-synthesize adversarial queries → RL query pool + query generation.

The three paths differ in output granularity: the first two repair individual trajectories one by one (producing SFT supervision); the third batch-generates tasks for an entire failure class (producing RL exploration directions).

Step-Level Correction: Repairing Occasional Single-Step Slips. When the trajectory prefix is entirely correct and only one step went wrong (tapped the wrong element, accidentally triggered a popup), with the correct action at that step determinable offline without depending on subsequent real-device state, we truncate the first $N - 1$ steps as the “prefix success state” and repair step N offline through a two-tier mechanism: as the primary track, the erroneous step’s context (screenshot, goal, action history) is fed to a stronger offline Teacher model, which rewrites the correct action for that step; as the fallback, cases with low Teacher confidence or hard scenarios (risk control, complex account states, or severe UI deformation) are routed to human repair. *Example.* A trajectory triggers a popup at step N while steps $1, \dots, N - 1$ are correct; we keep that prefix and let the Teacher model rewrite step N into the correct action, yielding a “prefix + single-step correction” label.

Because the real-device state cannot be restored, the repaired action cannot be re-executed and verified on the original trajectory; this path therefore produces a “prefix trajectory + single-step correction label,” not a complete post-recovery trajectory. Its supervision value lies in letting the model learn the correct action within the real erroneous-state context of “having already gone off track”—single-step supervision on “how to correct after a mistake.” A complete multi-step “deviate–diagnose–recover” trajectory cannot be constructed offline and can only be re-obtained on real devices in the next round of rollout.

Retroactive Query Alignment (RQA): Downgrading an Over-Scoped Task into the Prefix’s Correct Solution. Sometimes the trajectory prefix is entirely correct, yet the root cause of failure is not “a wrong step”—the original query was over-scoped, exceeding the current model’s capability or containing unreachable subgoals. The prefix is not wrong; what is wrong is the goal it serves. RQA adopts a “subtract only, never redo” offline extraction strategy, retroactively downgrading the over-scoped query into a subtask that precisely fits the completed prefix. When the trajectory is detected to deviate from the goal or hit an environment error at step t , the actions a_t and beyond are dropped, keeping the entirely correct sequence $a_1, \dots, a_{t-1}$ as the “golden prefix sub-trajectory”; the screenshot s_{t-1} and layout tree of the last frame before the error are extracted to parse the actual page state; and an offline Teacher model rewrites the original instruction Q into a new instruction Q' that precisely matches the terminal-state semantics of s_{t-1} . The reconstructed pair $(Q', [a_1, \dots, a_{t-1}])$ is then injected into the SFT pool as a fully correct short-chain sample, with zero resampling cost.

Example. The original instruction Q is “search for the Sony XM5 headphones on Taobao, also open JD.com to compare prices, and place the order on whichever platform is cheaper.” The model completed the Taobao search, the JD.com search, and the price comparison, but failed at final checkout because no payment method was bound. RQA truncates the checkout segment, keeps the “dual-platform search + price comparison” prefix, and rewrites Q into Q' : “search for the Sony XM5 headphones on Taobao and JD.com respectively, compare prices across the two platforms, and note which one is cheaper.”

In short, step-level correction assumes “the task is right; one step went wrong,” so it changes the action; RQA assumes “the actions are all right; the task was over-scoped,” so it changes the task.

Counterfactual Query Derivation: Turning a Class of Failures into Targeted Training Tasks. The first two paths handle individual trajectories case by case, but a capability gap cannot be fixed by case-by-case repair. After clustering failed trajectories by structured attribution (§2.5.1), if the same error class recurs across multiple trajectories and multiple queries—e.g., “multi-constraint filtering settled early with only some conditions satisfied,” “information extracted on a previous page forgotten

after cross-page operations,” “tapped Cancel on a system confirmation dialog”—what it points to is a capability deficiency of the model (corresponding to a capability-profile dimension in §2.2.4), and systematic reinforcement is needed. The ECDM first induces the structured features of the failure class (triggering scenarios, target operation flows, the missing interaction intuition); the Teacher model then batch-synthesizes adversarial new queries that precisely target the gap based on the failure pattern. *Example.* For “missing conditions in multi-constraint filtering,” it generates “find a restaurant rated above 4.5, with per-person spend under 100 CNY, and delivery support—all three conditions are mandatory”; for “cross-page forgetting,” “remember the price at the first store, compare it at the third store, and order from the cheapest.” This batch of adversarial queries is injected into the next round’s RL query pool and simultaneously fed back into query generation, forcing the model to repeatedly explore at its fragile capability boundary until it converges to the correct policy.

Step-level correction and RQA recover usable supervision from failed trajectories, but only for existing capabilities; counterfactual derivation turns failure patterns to advantage, producing new tasks that attack the gaps—the former repairs data, the latter generates direction.

2.5.3. Lifecycle of the Self-Evolution Loop

The self-evolution loop is not endless mechanical repetition. As iterations proceed, the loop exhibits qualitatively distinguishable characteristics across lifecycle stages, with model capability and corpus quality rising steadily together:

- **Early stage · Behavioral cloning and norm establishment:** SFT-dominant. Through the high-purity gold-standard data produced by HTSC, the model rapidly absorbs GUI fundamentals—layout, element localization, and basic interaction norms—building basic task-execution intuition.
- **Middle stage · Exploration-boundary widening and robustness strengthening:** the RL share rises significantly. Large-scale real-device rollouts shape fault tolerance through dynamic rewards; the ECDM batch-revives failed trajectories (step-level correction + RQA) and generates adversarial derived queries at high frequency, feeding back into query generation and forcing the model to self-calibrate in a variable, noisy real-device environment. The model’s anti-interference and anomaly self-healing abilities improve markedly in this stage.
- **Late stage · Dynamic balance and self-refinement:** queries corresponding to simple tasks, having sustained high pass@K or reward, are successively removed from the RL active set as mastered samples, and the RL query pool automatically concentrates into the model’s hardest capability boundary. Every time the model conquers a high-difficulty long-tail case, the overall difficulty of the corpus shifts up once—forming a self-evolution loop of “stronger model → deeper boundaries discovered → harder derived corpus → even stronger model.”

3. Model Training

We train BlueLM-GUI from a general VLM in three stages. Continual pre-training (CPT) uses a large-scale uncurated training set and teaches the model to act over a sequence of screens. Supervised fine-tuning (SFT) uses expert-verified trajectories to make each decision accurate. Both stages consume samples from our data generation pipeline. Reinforcement learning (RL) then releases the model from imitation: guided by the principle *Every Rollout Is Real*, every exploration step runs in an online environment and receives a real response, so the capability the model acquires naturally fits deployment. After each rollout, the trajectory flows back into the data pool, turning the flywheel. Fig. 5 summarizes the three training stages and the capabilities each stage is designed to impart.

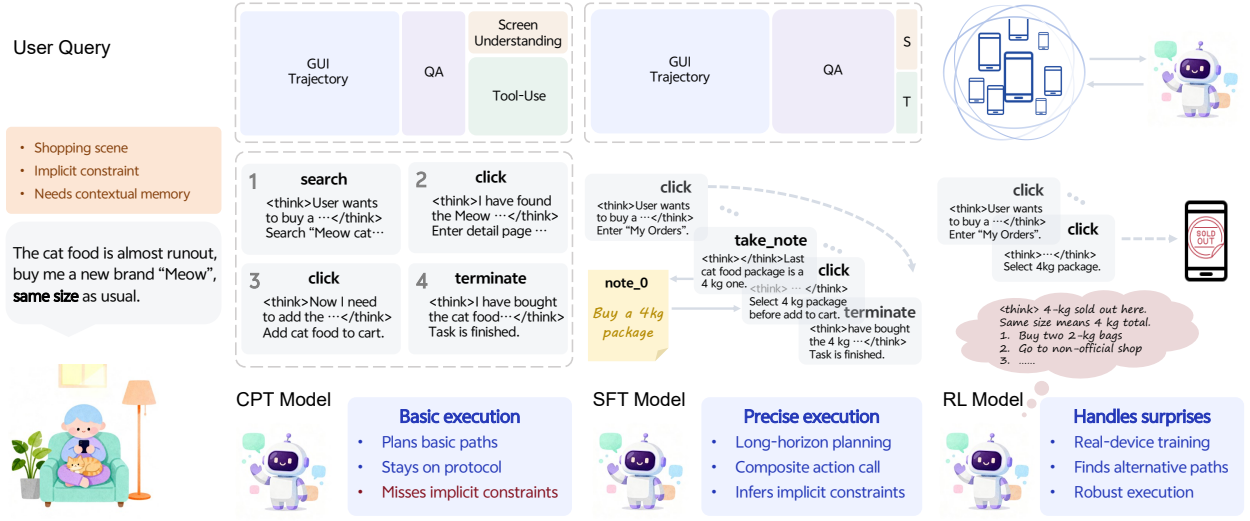


Figure 5 | The three training stages of BlueLM-GUI. CPT learns multi-step planning over a sequence of screens. SFT learns to follow implicit constraints and call composite actions. RL trains on real devices, gaining robustness and learning to handle real-world surprises.

3.1. Continual Pre-training

Base models nowadays understand screen elements well, but they are weak at predicting the next action in a trajectory, where a decision needs the model to track state across steps. What they lack is not knowledge about interfaces but a prior over sequential decisions: which stage of the task is active, whether the last action worked, or whether to move forward or back. However, keeping the SFT recipe fixed, SFT on the base model stays well below SFT after CPT (§5.3.1). CPT and verified data therefore do different jobs. CPT moves a general VLM into the GUI domain, and it does so at a per-sample cost far below that of verified data.

3.1.1. Training Corpus and Objective

The CPT corpus holds several million samples. Most of it is GUI trajectories, each a sequence of screenshots paired with the model outputs for those screens; §2 describes how we collect and filter them. Three smaller sets protect abilities the base model already has. Screen understanding data keeps grounding accuracy from drifting. Tool-use data helps because GUI task operates in a similar way. We also introduce general question-answering and reasoning data, which keeps the reasoning ability that goal decomposition and error handling depend on.

We use standard autoregressive language modeling, update all language-model parameters and freeze the vision encoder and the projector. Let y_t be the response at step t and x_t be its input context, the loss is calculated in the following function:

$$\mathcal{L}_{\text{CPT}}(\theta) = -\mathbb{E}_{(x_t, y_t) \sim \mathcal{D}_{\text{CPT}}} \left[\sum_{j=1}^{|y_t|} \log \pi_{\theta}(y_t^j | y_t^{<j}, x_t) \right]. \quad (1)$$

3.1.2. Interaction Protocol

This stage also fixes how the model talks to the environment: one input format, one output format, one action space. Later stages reuse all these three. The input at step t has four parts: a system prompt $\mathcal{P}$ that lists the action space and the rules to follow, the user goal g in natural language, the

Table 3 | The action space $\mathcal{A}$: fourteen actions in four groups.

Group	Action	Behavior
Tap-level action	click	Tap at coordinate (x, y) .
	double_click	Tap twice quickly, for zooming or text selection.
	type	Enter text into the widget at a given coordinate.
	system_button	Press a device key such as menu, enter, or screenshot.
Gesture-level action	swipe	Slide between two coordinates to scroll or turn a page.
	drag	Move an element to a target coordinate and release.
	long_press	Hold to open a context menu or enter an editing state.
Composite capability	open	Declare a functional need and let the device resolve it to an installed app.
	set_scroll_widget	Set a wheel picker to a target value in one call.
	take_notes	Save useful screen content to memory; credentials and card numbers are not allowed.
Control and handoff	wait	Skip one step while a page loads.
	terminate	End the trajectory with success or failure.
	clarify	Ask a question or confirmation, with optional choices.
	call_user	Pause and hand over control, with a reason.

interaction history $\mathcal{H}_t = \{(c_i, a_i)\}_{i=1}^{t-1}$, and the visual observations o_t of the screen. The output at each step is a text segment c_t , holding the reasoning trace and a summary of progress, plus one structured action $a_t \in \mathcal{A}$ that the device executes:

$$(c_t, a_t) \sim \pi_\theta(\cdot \mid \mathcal{P}, g, \mathcal{H}_t, o_{\max(1, t-K+1):t}), \quad (2)$$

where o_t is the screen after a_{t-1} ran and before we decide a_t , and K is the size of the visual context window. We treat the two kinds of memory differently on purpose. We keep all past text, which holds the model to the same goal over a long task. We keep only the last K screenshots, which keeps context length and inference cost affordable.

The action space $\mathcal{A}$ has fourteen actions in four groups (Tab. 3). The first two groups are primitive actions, split by whether a touch is discrete or sustained: tap-level actions are single-point touches and key presses, while gesture-level actions involve continuous contact like swiping and dragging. The third group contains composite capabilities, each of which folds a long primitive sequence into a single intent-level call. The fourth group covers control and handoff: actions that pause the trajectory, end it, or hand authority back to the user. Putting all four groups in one space means the model does not choose between two modes; it weighs every option in one decision. After CPT the model follows the protocol reliably and emits legal calls with a stable format across steps.

3.2. Supervised Fine-tuning

After CPT the model makes progress on tasks and respects the protocol. But a legal trajectory is not always the right one or the short one, and the model still breaks rules from time to time. SFT fixes both: at every step we align the model with a verified decision and with the rules it was given. We change the data mix to match. GUI trajectories take a larger share, and we draw them only from the high-confidence pool built by cross-model voting and expert verification (§2.4). We cut screen understanding and tool-use data to the smallest amount that keeps those abilities alive. We keep a large share of general reasoning data, because goal decomposition and reading implicit constraints rely on it. The training setup and objective are the same as in CPT.

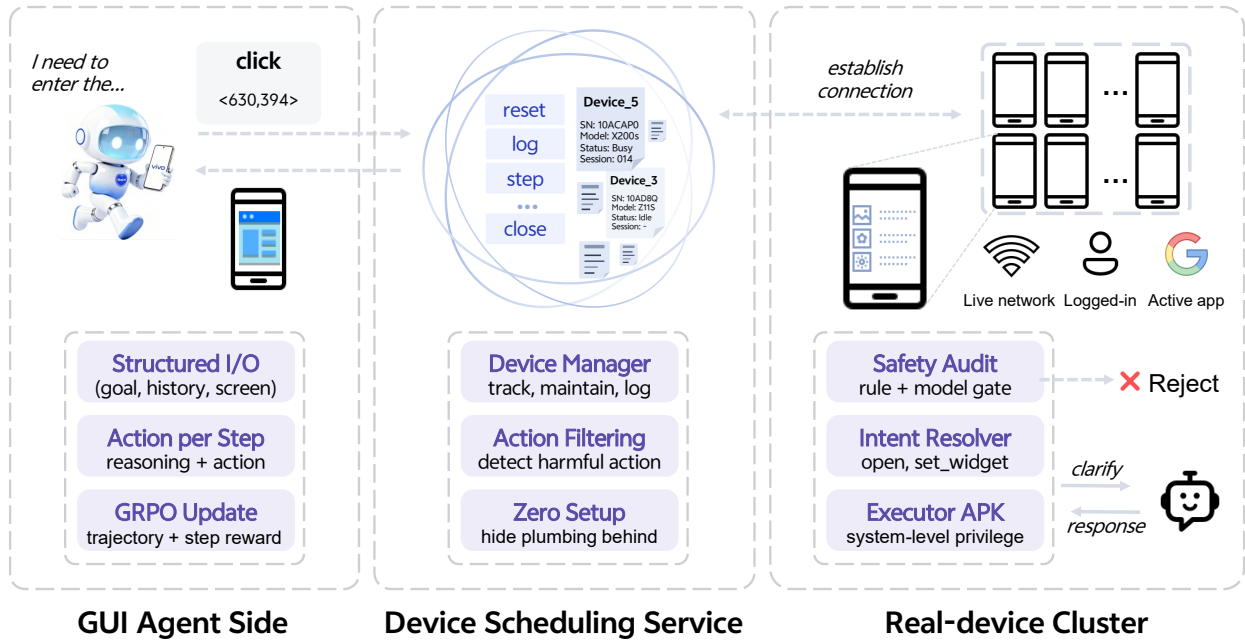


Figure 6 | The real-device training environment. A scheduling service gives the trainer a standard RL interface over a cluster of real devices.

3.2.1. Intent-Level Actions

The second goal of this stage is to let the model act at the level of intent rather than of primitives. Primitives are easy to learn, but they come with longer sequences, more time on device, and more risk. So we fold common primitive sequences into actions defined by intent. Take launching an app as an example. Many apps can meet the same need, but which ones are installed depends on the phone, the region, and user preference. With primitives only, the model has to try candidates one by one. We therefore let open take a description of the need instead of a fixed app name, and the device resolves that description to an installed app. In this way, the model decides what to do next; the device works out how to do it. And other composite capability actions follow the same pattern.

3.2.2. Behavioral Boundaries

If the instruction itself is unclear, or if the model reaches a boundary it must not cross on its own, the model has to involve the user. Two actions cover these cases. `clarify` collects what is missing into one question and resumes with the session intact once the user answers. `call_user` stops at the critical screen, hands control back, and supplies the information the user needs to decide.

Which action applies depends on what is missing. `clarify` handles a gap in the instruction, where the model cannot tell which of several readings the user meant. `call_user` handles a consequence the model is not entitled to accept on the user's behalf, such as a payment or a deletion. Both actions already exist after CPT, what SFT teaches the model is the timing: when to ask the user, and when to go ahead on its own.

3.3. Agentic Reinforcement Learning

SFT can only teach the model to reproduce the trajectories we annotated, and the annotations can cover only a fraction of the obstacles a real device will throw at it. Learning to recover from unforeseen

obstacles requires the model to try its own solutions and see how they turn out. This rules out scoring a fixed set of collected trajectories, because a GUI task usually has multiple valid paths and a reward defined on a single one of them would reject the others. We therefore perform online training: the model acts in an online environment, and the reward is computed from the actual resulting state.

3.3.1. Real-Device Environment

Our environment has three layers (Fig. 6). The model emits actions in the form of Eq. 2. A scheduling service exposes the usual reinforcement learning interface of reset, step, and close, and hides everything below it, so the trainer can rollout through many phones at once. The bottom layer is a cluster of hundreds of phones of different models, each running a resident runtime that executes an action and returns the new screen.

The phones in our cluster carry up-to-date apps, live network, and signed-in accounts, so the model meets real pop-ups and real loading delays during training rather than after release. However, on the other hand, free exploration on such devices is risky, so every action passes through a safety audit system that blocks actions involving private data, money, or prohibited content. This system is the one we ship in production, so the same safeguard that protects our devices during training stays in place after release. The phones also run a local harness that expands composite capability actions into concrete operations, keeping the action space identical across training and deployment.

3.3.2. Reward Design

We train on tasks from the middle difficulty band of our data engine, since tasks that every model solves or every model fails produce no useful gradient. An LLM judge scores each trajectory τ from the goal, the full sequence of outputs and actions, and the screen at every step, so the score reflects both the decisions and their effect on the device. We query the judge multiple times and take a majority vote. The trajectory reward then combines *progress* and *quality* at equal weight:

$$R(\tau) = \underbrace{\alpha \cdot \frac{N_{\text{done}}}{N_{\text{nec}}}}_{\text{progress}} + (1 - \alpha) \cdot \underbrace{\frac{N_{\text{nec}}}{T}}_{\text{quality}}, \quad \alpha = 0.5, \quad (3)$$

Here N_{nec} is the number of sub-goals the judge considers necessary for the task, N_{done} is how many of them the trajectory reached, and T is the number of steps it took. Progress rewards the model for getting as far as it can, while quality measures how many of its steps were actually needed, which prevents it from reaching the goal through a long detour.

A single reward per trajectory tells the model that an attempt failed but cannot tell which step t was responsible for it. We therefore also ask the judge to rate every action,

$$r_t = \text{Judge}(g, \tau, t) \in \{+1, 0, -1\}, \quad (4)$$

where the three values correspond to a step that is necessary, a step that is harmless but useless, and a step that pushes the task off course.

3.3.3. Training Optimization

We optimize with Group Relative Policy Optimization (GRPO). For each task we sample G trajectories, normalize the two rewards separately, and add them into a single advantage:

$$A_{i,t} = \frac{R(\tau_i) - \text{mean}(\{R(\tau_j)\}_{j=1}^G)}{\text{std}(\{R(\tau_j)\}_{j=1}^G)} + \lambda \cdot \frac{r_{i,t} - \text{mean}(\{r_{i,t'}\}_{t'=1}^{T_i})}{\text{std}(\{r_{i,t'}\}_{t'=1}^{T_i})}. \quad (5)$$

The trajectory reward is normalized within the group of one task, where it compares whole solutions against each other, and the step reward is normalized within one trajectory, where it compares individual steps. The weight λ balances the two. The policy objective is

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau_i \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{T_i} \sum_{t=1}^{T_i} \min \left(\rho_{i,t} A_{i,t}, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) A_{i,t} \right) \right], \quad (6)$$

where $\rho_{i,t}$ is the probability ratio between the current and the sampling policy. We also add a KL term toward the SFT model, which limits how far exploration can move the policy and protects the protocol and the safety boundaries learned earlier.

Additionally, we adjust the rollout schedule during training: groups whose success rate falls outside a useful range are discarded because they carry no contrast, and when sampling fails we fall back on earlier successful trajectories of the same task, with a cap on how much of an update they may contribute. Every trajectory we collect is kept and returned to the data pool, where verified successes become supervision for later runs and failures are routed through cascaded attribution into the ECDM. This closes the loop with the data engine of §2.5, and lets the model keep improving across iterations instead of finishing in one pass.

4. MobileGUI-VBench

Every Query Evolves. A fixed benchmark saturates as BlueLM-GUI improves and the scores stop telling us what to fix. What we need is therefore a benchmark that can evolve under a fixed metric system. Our three axes—scenario, complexity, and interaction & risk—serve as a stable coordinate system. When one benchmark saturates, we shift the quotas along a few dimensions and obtain a new one that differs from the old only in an explicitly stated way, so every score change is attributable. Our construction pipeline consists of five steps. First, we define a set of annotation dimensions that turns the question of what to test into a question of how to allocate quotas (§4.1). Second, we write the purpose of a new benchmark as a target quota over those dimensions. Third, an LLM generates queries against the quota. Fourth, experts verify and refine every query before it ships (§4.2). Fifth, we score each trajectory through independent multi-judge voting (§4.3).

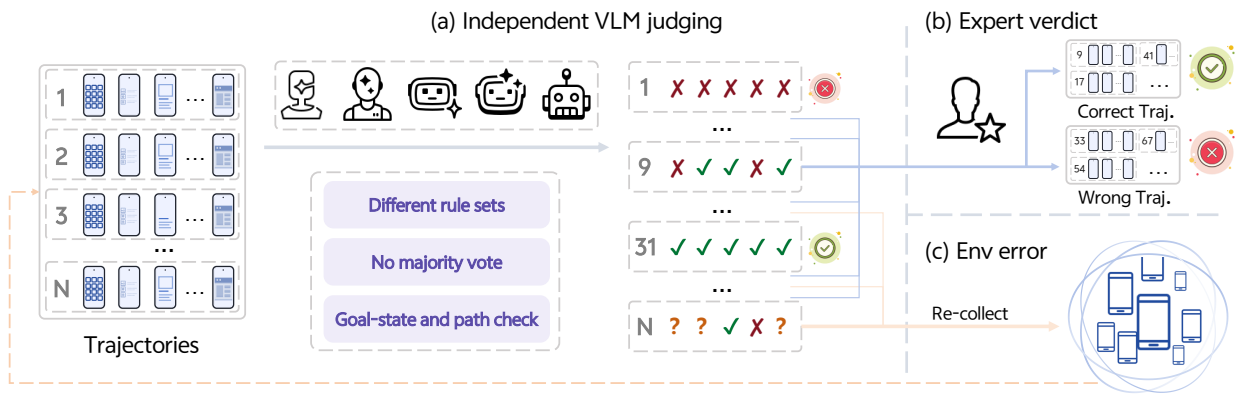


Figure 7 | The evaluation protocol. A trajectory is scored by multiple independent LLM judges. When all judges agree, the verdict stands; when they split, the case goes to an expert verdict. Trajectories broken by the environment are re-collected.

4.1. A Quantified Metric System

Our annotation dimensions exist so that queries can be produced in bulk to a specification. Tab. 4 lists them. They form three orthogonal axes, which ask where the task happens, how hard the execution is, and what form the interaction takes. Every value is a discrete level, so any benchmark could be defined by one set of quotas over this space.

4.1.1. Scenario Metric

The top-level split is between system apps and third-party apps, and it reflects two demands we cannot merge. Our goal is the end-user experience, where a task in Settings or Gallery matters as much as one in a shopping or navigation app. However, comparing models on system apps would nevertheless be unfair, since system apps are designed differently. Everything we report and compare in this work therefore uses third-party apps.

Below this split we spread queries over twelve app categories, such as shopping, video, and messaging, so that no small group of apps dominates the benchmark. The finest level is the functional entry point. Two functions inside one app might differ more than two apps do, and a quota defined only over apps tends to keep hitting shallow functions on the main pages of apps. We therefore run an automated discovery of in-app functionalities and use the entry points it finds as our candidate pool. Coverage then follows the functions an app actually offers, not how well the query author happens to know that app.

4.1.2. Complexity Metric

The main measure here is path complexity, which we define by the number of GUI steps a task needs and record as simple, moderate, or complex. One constraint decides whether this label is worth anything: the step count cannot be derived from the wording of the instruction. *Book me a ticket* is four words long and can take more than ten steps on a real interface. We therefore have an LLM walk through the likely path for each query, and we correct its estimates during expert refinement.

Orthogonal to path complexity is intent dependency, with three values: single-intent, independent multi-intent ($A + B$), and dependent multi-intent ($A \rightarrow B$). In independent multi-intent ($A + B$), a query bundles several sub-tasks that can be completed in any order without passing results between them. And in dependent multi-intent ($A \rightarrow B$), the model must feed the output of one sub-task into the next, and a query can demand this without being long. We also record app scope, which is single-app or cross-app, and operation type, which is tap-level or gesture-level.

4.1.3. Interaction and Risk Metric

The first two axes assume that a task should be carried out. But real use does not work all that way, and this axis covers what lies outside plain execution. We set instruction explicitness that has three values: an explicit instruction, an implicit intent that names a function but leaves out the path, and an underspecified instruction that states a problem without a goal. Instruction type separates agent-executable queries from agent-user interactive ones, where the model is supposed to stop at a decision point and ask rather than decide for the user. Refusal and safety covers requests an app cannot support: requests beyond the capability scope of the agent, commonsense-violating requests, and high-sensitivity operations. For these queries the correct behavior is to refuse or to clarify. Without them, a model that acts regardless of the request would score higher than one that knows its limits.

Table 4 | Our metric system. Three orthogonal axes cover where a task happens, how hard it is to execute, and what form the interaction takes. All values are discrete, so a benchmark is defined by a set of quotas over this space.

Axis	Dimension	Values	What it tests
Scenario	App ownership	system / third-party	On-device experience; fair comparison across models
	App category	12 categories, e.g. shopping, video, messaging	Generalization across scenarios
	Functional intent	varying across apps	Reachability of deep functions
Complexity	Path complexity	simple ≤ 5 / moderate 6–10 / complex > 10 steps	Long-horizon execution and progress tracking
	Intent dependency	single / independent ($A + B$) / dependent ($A \rightarrow B$)	Task decomposition and passing results across steps
	App scope	single-app / cross-app	Cross-app state transfer
	Operation type	tap-level / gesture-level	Fine-grained gestures
Interaction & risk	Explicitness	explicit / implicit / underspecified	Intent understanding and commonsense completion
	Instruction type	agent-executable / agent-user interactive	When to ask instead of deciding for the user
	Refusal and safety	unsupported / beyond scope / non-GUI / high-sensitivity	Capability-boundary awareness and risk avoidance

4.2. Benchmark Construction

The first output of construction is a distribution table. It states which direction generation should lean and which scenarios may not be missing, and it lets two benchmarks built on the same dimensions be compared directly. We built two. MobileGUI-VBench-Basic answers whether the model is usable at all, and MobileGUI-VBench answers where the model still falls short. It holds 150 queries over 39 apps, and its quota is not a scaled-up copy of MobileGUI-VBench-Basic but a deliberate skew along a few directions. Fig. 8 shows the resulting composition.

4.2.1. Scenario Quotas

MobileGUI-VBench-Basic has to answer whether the model works on the phone a user actually holds, so its scenario quota follows app popularity rather than an even grid. Its current version spans 43 apps, 34 third-party and 9 system ones, which together cover 36 of the 50 most installed titles. The system apps are universal utilities such as Messaging, Gallery, and Calculator. Eleven categories are represented, five of which take 12 to 16 percent each while the rest form a long tail.

MobileGUI-VBench uses third-party apps only, which makes its scores fair to compare against other models. Within that scope we keep the major categories, such as shopping and video, at similar quota, and we retain a few long-tail categories such as news reading, so that no single interface style drives the conclusions. We also cap how many queries any one app may contribute, so that the implementation quality of one app cannot hold the whole score hostage.

4.2.2. Complexity Quotas

MobileGUI-VBench-Basic allots path complexity roughly as 3 : 4 : 2 over simple, moderate, and complex. What the quota spends least on is the combination of a short path and a rarely used function, since a benchmark meant to certify usability learns little from it.

MobileGUI-VBench keeps moderate paths as the main body but cuts simple ones further, retaining only a few as a lower anchor, since they no longer separate models. Its added difficulty comes less from longer paths than from intent dependency, whose dependent multi-intent share we raised sharply. We weighted this direction because it tests directly whether the model can pass the result of one sub-task into the next. We also raised the share of gesture-level operations somewhat, while keeping tap-level operations in the majority, as they remain the common case in real use.

4.2.3. Interaction and Risk Quotas

MobileGUI-VBench-Basic sets no quota for refusal and safety. Its agent-user interactive queries already test when the model should pause and ask rather than decide, but the benchmark never asks it to refuse an operation. A model that never says no could therefore score well on MobileGUI-VBench-Basic while being reckless in practice. MobileGUI-VBench was built to close this gap.

The other structural change in MobileGUI-VBench is that agent-user interactive queries now make up close to 40 percent of the benchmark. On a real phone, deciding for the user at a point that needed confirmation is usually worse than pausing to ask. Giving these queries a large share rewrites the standard of success from whether the operation was completed to whether the model responded appropriately at the right moment. Risk scenarios follow a different rule: rather than getting a proportional share, they must simply be present. Queries that touch money, private data, or any irreversible change belong in the benchmark however small their share, because otherwise a model that acts regardless of the request scores higher than it should. We did not invent these quotas either. We collected failure cases from our own models, picked out the scenarios they got wrong, and generalized from those, so that the skew points at real weaknesses.

4.2.4. From Quotas to Queries

Once the quota table is fixed, we turn the dimension definitions and the annotation rules into structured constraints, and an LLM generates queries against the target distribution while annotating them at the same time. We validate every field as it is produced, then compare the realized distribution against the quota and patch the parts that drift. What the model produces is a draft. Before a benchmark ships, experts go through it query by query: they rewrite stiff phrasing into what a user would actually say, correct wrong labels, and attach to each query a reproducible description of the environment it assumes, marking whether that condition holds once or has to be reset for every run. This last item is what makes results comparable at all. Unless we know which parts of the device state must be restored, the same query run twice, or run on two models, is not the same query.

4.3. Evaluation Protocol

One evaluation of a GUI agent is a trajectory that actually happens on a phone, so we report a single metric: the success rate, the fraction of queries on which the model completes the whole task by itself. Writing Q for the benchmark and τ_q for the trajectory the model produces on query q ,

$$\text{SR} = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{1}[\text{Judge}(q, \tau_q) = \text{success}], \quad (7)$$

where Judge returns a binary verdict and we award no partial credit. We give no credit for partial paths because on a real device getting most of the way there is worth nothing to the user. The same rule decides the other two kinds of query. On an agent-user interactive query, a model that passes a decision point involving money or private data without calling `call_user` fails, even if the task appears to have been completed. On a refusal query, refusing or clarifying correctly counts as success.

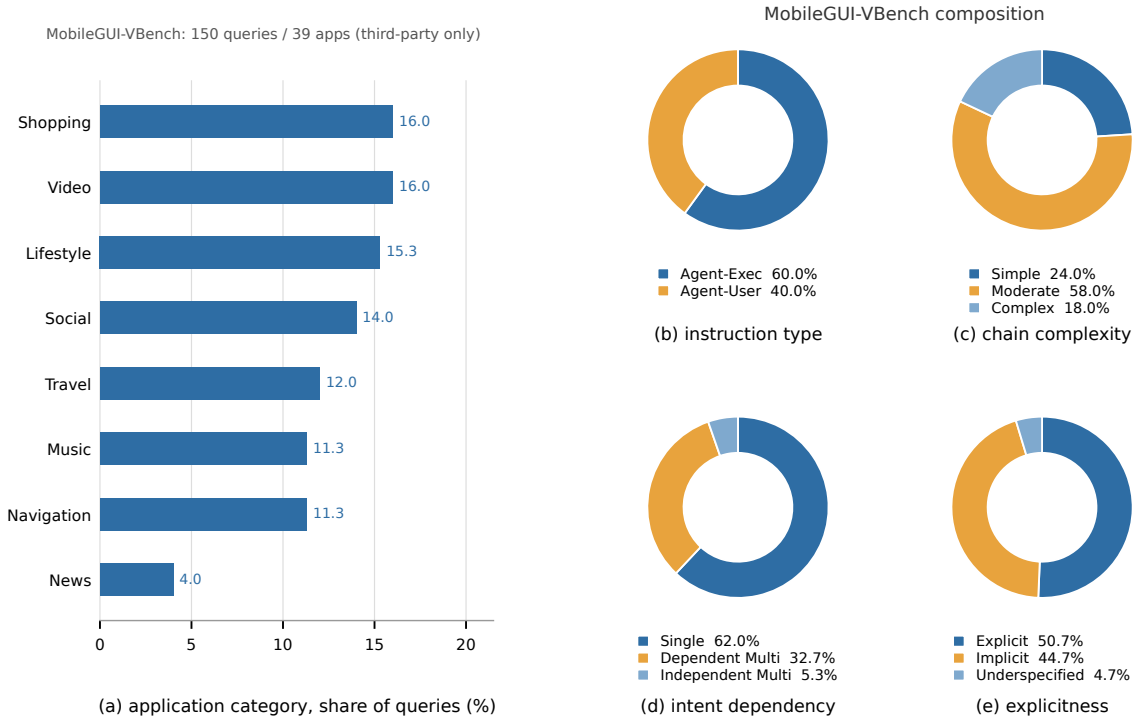


Figure 8 | Composition of MobileGUI-VBench. The bar chart gives the distribution over app categories, and the rings give the distribution over path complexity, intent dependency, instruction explicitness, and instruction type.

A single metric thus constrains execution and interaction behavior at once, and a model cannot trade recklessness for score.

Judging is the hard part, because deciding whether a trajectory reached its goal usually means reading the semantics of the last screen and often of several screens before it. We use several VLM judges, fine-tuned for this task, with different rule sets, each ruling independently on whether the goal state was reached and whether the model interacted at the right moments, and we count a trajectory as successful only when all of them agree. This unanimity requirement is deliberate, because it turns disagreement among judges into a signal: a trajectory the judges split on lies outside the range where automatic judging is reliable. We export those cases for experts to score by hand, and we use their verdicts to fix whatever was ambiguous in the judging rules.

Real-device evaluation also suffers from failures that have nothing to do with the model, such as an expired login, a redesigned interface, or a network error. Counting these as failures would mix unrelated variance into the metric. We therefore attribute every failure first, and a trajectory broken by the environment is not scored. We reset the device and collect it again.

5. Experiments

5.1. Training Details

All models start from the same base model, Qwen3.5-35B-A3B, and pass through CPT, SFT, and RL in that order, with each stage continuing from the output of the previous one.

Continual pre-training. We train our model in the CPT stage on 64 GPUs. The training corpus, consisting of 15B tokens, draws on four complementary sources, roughly 70 percent is GUI trajectories data, others include screen understanding, question answering, and public tool-use data. Both GUI and screen data are collected from real devices. We freeze the vision encoder and the projector and update only the language model, so that the model picks up a temporal decision prior while keeping the perception ability of the base model. The learning rate is 1×10^{-5} with a warmup fraction of 0.05 and cosine decay to 1×10^{-6} , the batch size is 128, the maximum sequence length is 10240, and we train for one epoch.

Supervised fine-tuning. SFT training uses the same freezing strategy on 32 GPUs and is conducted on a 5B-token corpus. GUI trajectories at this stage are collected on real devices and verified by experts, so the corpus is smaller than the CPT one but far higher in annotation quality and task complexity. The learning rate and the schedule follow CPT, the batch size rises to 160, the maximum sequence length is 10240, and we train for one epoch.

Reinforcement learning. RL is optimized via GRPO on 32 GPUs, interacting with an environment consisting of hundreds of real devices and Android emulators. The training instruction set holds over a thousand queries. Each iteration samples 8 instructions and rolls out 8 trajectories per instruction, and these 8 trajectories form the group over which we compute advantages. The weight λ is set to 1 by default. The learning rate drops to 1×10^{-6} and the batch contains 512 interaction windows. A trajectory runs for at most 20 interaction steps, visual context window keeps most recent 5 frames, and the prompt is capped at 12288 tokens.

5.2. Comparison with State-of-the-Art Models

We compare BlueLM-GUI against the strongest agents we could reach, on two benchmarks: MobileGUI-VBench, and the public AndroidWorld. The baselines fall into three groups. Closed-source models reached through an API, closed-source models reported by their authors but not released, and open-source models. Every number in Tab. 5 is a success rate on a real device or in a live environment, so the comparison covers whole trajectories rather than single-step predictions.

BlueLM-GUI reaches 87.4 on MobileGUI-VBench with 35B total and 3B activated parameters. The best closed-source API model scores 82.3, and the best open-source baseline scores 66.4. We read this gap as a benchmark that tests what our metric system was built to test: most of the MobileGUI-VBench queries carry dependent multi-intent structure, or ask the model to stop and consult the user, or must be refused outright. A model tuned for straightforward execution has nowhere to earn those points. The spread within the baselines supports this. Several models that do well on AndroidWorld drop sharply here, and PhoneBuddy is the clearest case, at 83.2 on AndroidWorld against 39.0 on MobileGUI-VBench.

On AndroidWorld, BlueLM-GUI reaches 84.9, surpassing all open-source baselines and closed-source models, behind only Qwen3.8-Max (85.3) and Qwen3.8-Flash (84.5) — both of which activate far more parameters than we do. What we want to point out is not the ranking but the shape of it. The models ahead of us on AndroidWorld are 64.2 and 59.1 on MobileGUI-VBench, roughly 25 points below us, while we sit within 2 points of them on AndroidWorld. Strong execution therefore does not carry over to the interaction and refusal behavior that MobileGUI-VBench asks for, whereas a model trained for the latter keeps its execution ability intact.

Model	Access	Params	MobileGUI-VBench	AndroidWorld
<i>Closed Source</i>				
Seed2.1-Pro	API	🔒	82.3	72.4
Seed2.0-Pro	API	🔒	80.8	71.5
Seed2.0-Lite	API	🔒	77.5	71.6
Gemini3.7-Flash	API	🔒	78.3	80.1
Claude Opus 4.8	API	🔒	56.9	63.8
Qwen3.7-Plus	API	397B-A17B	49.6	81.0
UI-Venus-2.0-27B	Closed	27B	-	84.0
UI-Tars-2.0	Closed	230B-A23B	-	73.3
MAI-UI-235B-A22B	Closed	235B-A22B	-	76.7
Xiaomi-GUI-0-30B-A3B	Closed	30B-A3B	-	78.9
Step-GUI	Closed	8B	-	80.2
<i>Open Source</i>				
GLM5.3-Flash	API	320B-A18B	66.4	78.4
Qwen3.8-Max	API	2.4T-A95B	64.2	85.3
Qwen3.8-Flash	API	125B-A6B	59.1	84.5
Qwen3.8-27B	Open	27B	58.0	81.9
Qwen3.6-35B-A3B	Open	35B-A3B	40.7	73.3
Qwen3.6-27B	Open	27B	64.4	70.3
Qwen3.5-122B-A10B	Open	122B-A10B	57.3	66.4
Qwen3.5-35B-A3B	Open	35B-A3B	42.2	71.1
Qwen3.5-27B	Open	27B	53.7	64.2
GUI-Owl-1.5-32B-Think	Open	32B	47.1	71.6
UI-Venus-2.0-9B	Open	9B	49.6	80.2
UI-Venus-1.5-30B-A3B	Open	30B-A3B	24.1	77.6
PhoneBuddy	Open	4B	39.0	83.2
<i>Ours</i>				
BlueLM-GUI	Ours	35B-A3B	87.4	84.9

Table 5 | Comparison of models on MobileGUI-VBench and AndroidWorld benchmarks.

5.3. Ablation Studies

5.3.1. The Necessity of Continual Pre-training

To test whether CPT is needed at all, we fix the SFT configuration and change only what comes before it: whether the model first sees the large weakly supervised corpus. We evaluate on screen understanding and on trajectory execution. The first measures single-step grounding and semantics of interface elements, the second measures multi-step execution by success rate.

Tab. 6 shows that the base model already scores 95.48 and 94.34 on screen understanding, yet completes only 36.6 of the trajectories, even though its prompt contains examples of how to call actions. The bottleneck is therefore not reading the screen. What the base model lacks is a notion of the action space, a causal link between consecutive steps, and any tracking of how far the task has progressed. SFT raises the success rate to 64.1 while leaving screen understanding flat, which tells us that expert-verified trajectories supply exactly this missing decision ability. Adding CPT on top pushes the success rate to 70.8, at the cost of slight degradation on the screen understanding benchmarks. Since both variants use identical SFT data, those 6.7 points come from the prior that CPT installs.

Model	ScreenSpot-ZH	ScreenSpot-v2	MobileGUI-VBench-Basic
Base	95.48	94.34	36.6
Base + SFT	96.48	93.08	64.1
Base + CPT + SFT	96.13	91.90	70.8

Table 6 | CPT is necessary. Both variants share one base model and an identical SFT configuration, and differ only in whether CPT precedes SFT.

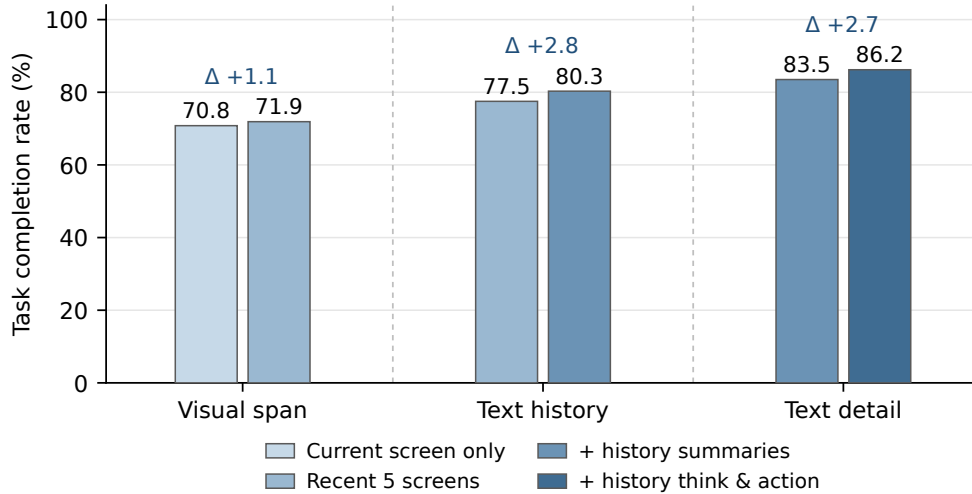


Figure 9 | Extending the interaction context raises the success rate. The three pairs sit on different baselines, so only within-pair differences are comparable.

When annotation budget is the binding constraint, building that prior from a large weakly supervised corpus is a path to improvement that does not require more annotation.

5.3.2. The Form of the Interaction Context

We next ask how the history should be organized for the model to use it. We extend what the model can see along two directions, visual and textual, in four settings: the current screen only, the five most recent screens, five screens plus the summaries of earlier steps, and five screens plus the full reasoning traces, actions, and summaries of earlier steps. Training and evaluation cost kept us from placing all four on one baseline. We instead ran three pairwise comparisons, each of which changes only the context form and holds everything else fixed, so the conclusions hold within a pair and should not be read across pairs.

The three pairs in Fig. 9 give +1.1, +2.8, and +2.7 points of success rate, so extending the history helps in every direction, but the visual and textual extensions are not worth the same. Going from one frame to five frames gives the smallest gain, +1.1. Earlier screens mostly supply continuity of appearance, while the information a decision actually needs, namely how far the task has gone and what the model has already tried, is hard to recover reliably from pixels. Stacking raw observations therefore does not give the agent the information it needs. The two textual extensions give about +2.8 each, both clearly above the visual side, so the model relies on history that has already been written down in language. The third pair also shows that reasoning traces and actions carry information the summaries alone do not, which is why we keep all three in the context rather than summaries only.

Mixture	ScreenSpot-ZH	ScreenSpot-v2	MobileGUI-VBench-Basic
Screen-heavy	96.13	91.90	70.8
Traj-heavy	96.60	90.41	71.9

Table 7 | Cutting screen understanding data and raising the trajectory share improves execution. Traj-heavy uses a CPT corpus 40% smaller.

Model	RL setting	MobileGUI-VBench-Basic
SFT	–	74.9
+ Offline RL	replay of collected trajectories	73.9
+ Online RL	real-device online interaction	82.0

Table 8 | RL needs a real-device environment. Both RL variants share one SFT starting point.

5.3.3. The Data Mixture across Stages

The base model is already strong enough at screen understanding tasks but weak at multi-step execution, which suggests that the mixture should tilt toward the latter. We test this by changing only the mixture, holding the data sources and the training procedure fixed: we cut the screen understanding data, which brings the total CPT corpus down by about 40%, and at the same time raise the trajectory share by a factor of four in CPT and two in SFT.

Tab. 7 shows the success rate rising from 70.8 to 71.9, while the two screen understanding scores move in opposite directions and stay flat overall. The gain arrives with a corpus 40% smaller, which is a win on both counts: we train faster and still execute better. Screen understanding data therefore reaches a point beyond which more of it no longer improves execution. The screen understanding scores also survive the cut, which means the base model already learned this ability during its general pre-training, and our training only has to adapt it to the interfaces our target apps use. So we design the mixture by verifying which abilities the base model already brings, giving those just enough data to transfer, and spending everything else on the abilities it lacks.

5.3.4. The Role of the Real-Device Online Environment

What RL returns depends on where the interaction data comes from. To separate the contribution of the algorithm from that of the environment, we start from one SFT model and apply two RL settings. The first optimizes on trajectories collected in advance, where the model’s outputs are never executed, so it never sees what its own choices lead to. The second samples in the real-device online environment, where every output is actually executed and returns the real next screen. Both use the same optimization algorithm, and the only difference is whether the interaction data is replayed or produced online.

Tab. 8 shows that the offline setting gains nothing over the SFT starting point, while the online setting raises the success rate to 82.0. Applying an RL objective to existing trajectories is therefore not enough. The state transitions in an offline trajectory were fixed, so once the current policy drifts away from that distribution, the advantage estimate no longer corresponds to any real consequence, and RL degenerates into reweighting the actions the dataset already contains. The real-device online environment closes the loop between an action and its consequence. The model sees how its own mistakes change the following state, and it finds workable paths that never appear in the annotated trajectories, which is where error recovery and multi-path solving come from. For long-horizon tasks whose state transitions depend heavily on how the environment responds, online interaction is not

Model	Judging	Passes	MobileGUI-VBench
RL-holistic	holistic, disjunctive	1	72.52
RL-atomic	atomic, frame-bound	5	77.42

Table 9 | Reward reliability drives RL. Atomic judging checks conditions that must all hold, each bound to a designated frame; Passes counts judging passes.

an optional refinement of RL but a precondition for it.

5.3.5. The Reliability of the Reward Signal

In online RL the update direction comes entirely from the reward, and a model-based judge is itself stochastic: the same trajectory can receive different verdicts on different passes. Group-relative advantages amplify it, because once a trajectory is scored too highly by mistake, the behavior it contains keeps getting reinforced. We therefore rebuilt the judging procedure. Instead of one holistic judgment of whether the task was completed, we write a set of atomic verification conditions that can be checked one by one, bind each condition to a designated frame, and require all of them to hold. This replaces the earlier disjunctive bypass rules, under which hitting any one condition was enough to pass. We also judge each trajectory five times to suppress single-pass randomness.

Tab. 9 shows the success rate rising from 72.52 to 77.42. Because the two runs share the same reward definition, those 4.9 points do not come from a better objective. They come from measuring the same objective more accurately. Online RL is thus limited by the judging step at least as much as by the optimization step. When the reward is noisy, sampling more or training longer can amplify a misleading signal, and making the judge more consistent does more good than tuning the algorithm.

6. Related Work

From Framework Agents to Native GUI Agents

Early GUI agents are typically constructed as multi-agent orchestration systems built upon general-purpose VLMs. Methods such as the Mobile-Agent series (Wang et al., 2024a) and AppAgent (Yang et al., 2023) compose specialized planner, executor, reflector, and memory modules around foundation models such as GPT-4o (OpenAI, 2024), Gemini (Gemini Team, 2024), or Qwen2-VL (Wang et al., 2024b). The performance of these approaches is largely bounded by the underlying VLM, and inference latency grows with module depth and the number of interaction rounds. Moreover, reflection resides in the runtime procedure rather than being internalized into model weights. To overcome these limitations, recent research has increasingly shifted toward native GUI agents, in which perception, grounding, planning, and execution capabilities are integrated into a single model through post-training. CogAgent (Hong et al., 2024), SeeClick (Cheng et al., 2024), OS-Atlas (Wu et al., 2024), and AutoGLM (Liu et al., 2024) established this paradigm, followed by UI-TARS (Qin et al., 2025a), UI-TARS-1.5 (ByteDance Seed, 2025a), and UI-TARS-2 (ByteDance Seed, 2025b), GUI-Owl in Mobile-Agent-v3 and Mobile-Agent-v3.5 (Xu et al., 2026; Ye et al., 2025), MAI-UI (Zhou et al., 2025), Step-GUI (Yan et al., 2025), UI-Venus-1.5 (Gao et al., 2026), PhoneBuddy (Tencent Hunyuan Team, 2026), Xiaomi GUI-0 (Cao et al., 2026), and HyMobileAgent (Shen et al., 2026). A parallel line extends the paradigm from mobile to general computer use, including MobileIPL (Huang et al., 2025), GTA1 (Yang et al., 2025a), OpenCUA (Wang et al., 2026a), UltraCUA (Yang et al., 2025b), Fara-7B (Awadallah et al., 2025), EvoCUA and EvoCUA-1.5 (Huang et al., 2026; Xue et al., 2026), CoME (Liu et al., 2026b), and ToolCUA (Hu et al., 2026). As native models mature, their capabilities

are increasingly determined by the data engines behind them: public corpora such as Android-in-the-Wild (Rawles et al., 2023), AndroidControl (Li et al., 2024), and AMEX (Chai et al., 2024) provide scale but remain success-biased; automated synthesis pipelines such as AUTO-Explorer (Guo et al., 2025), AndroidLab (Xu et al., 2024), GUI-Odyssey (Lu et al., 2024), OpenMobile (Chen et al., 2026), and UI-Genie (Xiao et al., 2025) reduce annotation cost through self-exploration, grounded instruction generation, and self-improving evaluation; and industrial systems have converged on closed-loop data engines—MAI-UI (Zhou et al., 2025) lets agents construct tasks, environments, and failure diagnoses, HyMobileAgent (Shen et al., 2026) scales a perception flywheel and a video-derived knowledge pipeline over thousands of sandbox and real-device instances, PhoneBuddy (Tencent Hunyuan Team, 2026) shows that mock-app training complements rather than replaces real-app data, and Xiaomi GUI-0 (Cao et al., 2026) recycles rollout failures into corrected actions and recovery demonstrations. Our work follows this native-agent line but adopts a distinct focus on two levels. First, rather than optimizing for static or emulator-dominated benchmarks, we target the constraints of real mobile deployment, including authentic account states, system dialogs, payment authentication, risk-control interventions, network variability, and client-side version drift. Second, whereas existing pipelines largely follow a “keep-clean-success, discard-failure” pattern, our data engine treats every sample as a source of supervision: we decouple CPT and SFT pools so that a single real-device collection pass yields both step-level perceptual supervision and trajectory-level planning supervision, and route failed and environment-error trajectories—collected exclusively on real devices so that long-tail conditions are covered—to a unified error-correction and derivation module for step-level correction, retroactive query alignment, and counterfactual derivation.

RL-based GUI Agent Training

DigiRL (Bai et al., 2024) is among the first to show that online reinforcement learning can substantially outperform supervised fine-tuning for device-control tasks, underscoring the value of interaction feedback. A wave of RL-style work follows, shifting the paradigm from purely offline imitation learning toward interactive policy optimization: UI-R1 (Lu et al., 2025), GUI-R1 (Xia et al., 2025), InfiGUI-R1 (Liu et al., 2025), and Mobile-R1 (Chen et al., 2025). Subsequent efforts scale RL to industrial-grade infrastructure: UI-TARS-2 (ByteDance Seed, 2025b) deploys thousands of parallel virtual machines to support large-scale RL rollouts, MobileRL (Wang et al., 2025) introduces adaptive GRPO under heavy-tailed task difficulty, MAI-UI (Zhou et al., 2025) supports online RL on trajectories exceeding one hundred turns with over ten thousand concurrent environments, and UI-Venus-1.5 (Gao et al., 2026) combines a four-stage pipeline of mid-training, offline RL, online RL, and model merging. A complementary line internalizes reasoning and self-correction into the policy itself: ReAct (Yao et al., 2023) and Reflexion (Shinn et al., 2023) establish the primitives of interleaved reasoning, action, and verbal self-critique; UI-TARS (Qin et al., 2025a) integrates a System-2 reasoning module before each action; and Xiaomi GUI-0 (Cao et al., 2026) turns failure trajectories into recovery demonstrations. Together, these studies establish reinforcement learning and built-in recovery as central directions for long-horizon GUI agents. Our training recipe builds on this line but closes the loop differently: rather than treating trajectories as either SFT demonstrations or RL rollouts with a one-way flow from data production to training, we route each trajectory according to its outcome—partially successful rollouts remain in the RL pool as the strongest exploration signal, fully failed rollouts flow to the ECDM and re-enter as SFT data, and only fully mastered queries retire from the pool—so that every trajectory continuously contributes to a co-evolving model–corpus curriculum.

Real-Device and Virtualized GUI Benchmarks

Existing mobile GUI benchmarks can be broadly divided into virtualized (emulator/sandbox) environments and real-device evaluations. GUI grounding is benchmarked by ScreenSpot (Cheng et al., 2024), ScreenSpot-Pro (Li et al., 2025), and MMBench-GUI (Qin et al., 2025b), while interactive evaluation is dominated by sandbox environments with programmatic verification: AndroidWorld (Rawles et al., 2025) provides execution-based grading on emulators, and OSWorld (Xie et al., 2024) and WebArena (Zhou et al., 2024) play analogous roles for desktop and browser. This sandbox line has since been extended primarily along coverage and task distribution: A3 (Chai et al., 2025), MobileBench-v2 (Xu et al., 2025), and PhoneWorld (Tang et al., 2026) broaden mobile platform coverage, while QwenClawBench (Qwen Team and Data Team, Alibaba Group, 2026) probes real-user task distributions. These settings offer scalable and reproducible evaluation, but their state distributions remain biased toward simplified environments and do not fully capture the account states, page dynamics, and business logic of real applications. To improve fidelity, simulation platforms such as MobileGym (Wu et al., 2026) and SimuWoB (Liu et al., 2026a) approximate the pages, databases, and partial functions of commercial applications, enabling scalable and verifiable benchmarking while remaining distinct from their real counterparts in state distributions and client-side behaviors. Real-device benchmarks, by contrast, evaluate agents directly on physical devices and live applications: MobileBench-OL (Zhao et al., 2025) measures task execution, reasoning, and noise robustness in an online real-device environment; MobileWorld (Kong et al., 2026) and AndroidDaily (Sui et al., 2026) target realistic daily-use scenarios; Xiaomi’s RealMobile (Cao et al., 2026) retains rule-based programmatic evaluation on live apps; and MAI-UI’s MobileWorld-Real (Zhou et al., 2025) introduces a multi-judge AutoJudge protocol for trajectory-level evaluation under live conditions. Our benchmark likewise runs on physical devices and live applications, but is designed so that every query is an accountable unit: queries are constructed with explicit functional-coverage indexing against app function trees, annotated with difficulty, ambiguity, and risk metadata, and evaluated with a protocol that separates model failure from environment failure and attributes residual disagreement to human adjudication—turning the benchmark from a static scorecard into a diagnostic instrument whose results feed back into data production and the RL query pool.

7. Conclusion

We have presented BlueLM-GUI, a 35B-A3B mobile GUI agent built as a real-device-centric flywheel for self-improvement. It targets three structural gaps that block industrial deployment, pairing each with a principle: the *distribution gap* between sandbox training and production is answered by *Every Rollout Is Real*, a three-stage CPT-SFT-RL recipe run end-to-end on hundreds of real phones; the *utilization gap*, in which costly real-device failures are discarded, by *Every Sample Matters*, an ECDM that salvages failed trajectories back into supervision; and the *attribution gap*, in which fixed benchmarks saturate, by *Every Query Evolves*, a quota-driven metric system whose benchmarks are upgraded as the model improves. BlueLM-GUI reaches 87.4 on MobileGUI-VBench, 5.1 points above the best closed-source API model, and 84.9 on AndroidWorld, competitive with closed-source models that activate far more parameters. Grounding the entire loop—training, salvage, and evaluation—in real devices is what makes the resulting capability transferable to deployment.

Looking ahead, we plan to scale the flywheel to broader device and distributions and to extend the interaction paradigm toward GUI + API + CLI fusion. Of these directions, the one most aligned with the self-evolution thesis is to reduce the human participation the loop still requires, moving it toward greater autonomy.

8. Contributors

Supervisors

- Xiaohu Ruan[†]
- Xiaoxin Chen

Contributors

- | | |
|------------------------------|------------------|
| • Tong Ye* | • Penggang Shi |
| • Kunyang Han* | • Mingcheng Luo |
| • Guozhi Wang* | • Ziyang Wu |
| • Longqiang Luo [†] | • Min Jin |
| • Zhifeng Ding | • Mingfu Shen |
| • Yongxiang Zhang | • Zairong Xu |
| • Xiaolei Shen | • Fan Zhang |
| • Yuxuan Zhang | • Hao Wang |
| • Zhuping Zhang | • Liang Liu |
| • Tao Xu | • Zhulin Xie |
| • Yue Pan | • Lijun Yao |
| • Yucheng Zhao | • Xiao Liang |
| • Yupei Hu | • Liangmin Wen |
| • Yuanjiang Ouyang | • Liqiang Feng |
| • Danfeng Shen | • Feilong Wu |
| • Runqi Lin | • Min Hu |
| • Hongda Cai | • Min Chen |
| • Zhaoxiong Wang | • Guanqing Xiong |
| • Mengjia Yan | |
| • Yingjie Zhong | |
| • Chen Zhou | |
| • Zeyu Zhang | |
| • Xuwen Zhu | |

* Co-first authors [†] Corresponding authors

References

- Anthropic. Introducing Claude Opus 4.8. <https://www.anthropic.com/news/claude-opus-4-8>, 2026.
- A. Awadallah, Y. Lara, R. Magazine, H. Mozannar, A. Nambi, Y. Pandya, A. Rajeswaran, C. Rosset, A. Taymanov, V. Vineet, et al. Fara-7b: An efficient agentic model for computer use. *arXiv preprint arXiv:2511.19663*, 2025.
- H. Bai, Y. Zhou, M. Cemri, J. Pan, A. Suhr, S. Levine, and A. Kumar. DigiRL: Training In-The-Wild Device-Control Agents with Autonomous Reinforcement Learning. *arXiv preprint arXiv:2406.11896*, 2024.

- ByteDance Seed. UI-TARS-1.5. <https://seed-tars.com/1.5>, 2025a.
- ByteDance Seed. UI-TARS-2 Technical Report: Advancing GUI Agent with Multi-Turn Reinforcement Learning. *arXiv preprint arXiv:2509.02544*, 2025b.
- W. Cao, C. Duan, P. Fu, P. Gao, N. Lian, F. Liu, H. Liu, H. Qu, Q. Wu, Z. Yu, et al. Xiaomi-GUI-0 Technical Report. *arXiv preprint arXiv:2606.31410*, 2026.
- Y. Chai, S. Liu, et al. AMEX: Android Multi-annotation Expo Dataset for Mobile GUI Agents. *arXiv preprint arXiv:2407.17490*, 2024.
- Y. Chai, H. Li, J. Zhang, L. Liu, G. Liu, G. Wang, S. Ren, S. Huang, and H. Li. A3: Android agent arena for mobile gui agents. *arXiv preprint arXiv:2501.01149*, 2025.
- J. Chen, Z. Li, T. Zhang, Q. Zhu, H. Li, X. Li, H. Zhu, H. Liu, Z. Xiao, X. Sun, and Z. Deng. Mobile-R1: Reinforced Interactive Mobile Agent via Task-Level Reasoning. *arXiv preprint arXiv:2506.20332*, 2025.
- K. Chen, S. Lu, et al. OpenMobile: An Open Data Synthesis Framework for Mobile Agents. *arXiv preprint arXiv:2604.15093*, 2026. URL <https://arxiv.org/abs/2604.15093>.
- K. Cheng, Q. Sun, Y. Chu, F. Xu, Y. Li, J. Zhang, and Z. Wu. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- DeepSeek-AI et al. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. *arXiv preprint arXiv:2606.19348*, 2026. URL <https://arxiv.org/abs/2606.19348>.
- Z. Gao et al. UI-Venus-1.5 Technical Report. *arXiv preprint arXiv:2602.09082*, 2026.
- Gemini Team. Gemini: A Family of Highly Capable Multimodal Models, 2024.
- Google DeepMind. Gemini 3.1 Pro Model Card. <https://deepmind.google/models/model-cards/gemini-3-1-pro/>, 2026.
- X. Guo, D. Gao, and M. Z. Shou. Auto-Explorer: Automated Data Collection for GUI Agent. *arXiv preprint arXiv:2511.06417*, 2025.
- W. Hong, W. Wang, Q. Lv, J. Xu, W. Yu, J. Ji, Y. Wang, Z. Wang, Y. Zhang, J. Li, et al. CogAgent: A Visual Language Model for GUI Agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- Z. Hou, Y. Li, J. Tang, and Y. Dong. Single-Rollout Asynchronous Optimization for Agentic Reinforcement Learning. *arXiv preprint arXiv:2607.07508*, 2026.
- X. Hu, X. Zhang, H. Xu, K. Qiao, J. Yang, X. Huang, J. Shao, M. Yan, and J. Ye. ToolCUA: Towards Optimal GUI-Tool Path Orchestration for Computer Use Agents. *arXiv preprint arXiv:2605.12481*, 2026.
- K. Huang, W. Xu, Y. Liu, Q. Wang, P. Gao, W. Liu, J. Luan, B. Wang, and B. An. MobileIPL: Enhancing mobile agents thinking process via iterative preference learning. *arXiv preprint arXiv:2505.12299*, 2025.
- M. Huang, T. Xue, C. Peng, J. Ding, S. Fan, J. Hong, Y. Gao, X. Zhang, L. Guo, X. Yang, et al. Evocua-1.5: Online reinforcement learning for multi-turn computer-use agents. *arXiv preprint arXiv:2607.09773*, 2026.

- Kimi Team et al. Kimi K3: Open Frontier Intelligence. *arXiv preprint arXiv:2607.24653*, 2026. URL <https://arxiv.org/abs/2607.24653>.
- Q. Kong, X. Zhang, Z. Yang, N. Gao, C. Liu, P. Tong, C. Cai, H. Zhou, J. Zhang, L. Chen, et al. MobileWorld: Benchmarking Autonomous Mobile Agents in Agent-User Interactive and MCP-Augmented Environments. *arXiv preprint arXiv:2512.19432*, 2026.
- C. Li, Z. Fang, Z. Tang, P. Lyu, X. Zhou, X. Lai, F. Tang, L. Wu, Y. Guo, W. Wang, et al. PhoneHarness: Harnessing Phone-Use Agents through Mixed GUI, CLI, and Tool Actions. *arXiv preprint arXiv:2606.14832*, 2026.
- K. Li, Z. Hu, et al. ScreenSpot-Pro: GUI grounding for professional high-resolution computer use. *arXiv preprint arXiv:2504.07981*, 2025.
- W. Li, W. Bishop, A. Li, C. Rawles, F. Appiah-Adu, et al. On the Effects of Data Scale on UI Control Agents. *arXiv preprint arXiv:2406.03679*, 2024.
- N. Lian, A. Chen, Z. Yu, C. Duan, F. Liu, H. Liu, P. Fu, J. Luan, Y. Wang, S.-T. Xia, and J. Wang. UI-MOPD: Multi-Platform On-Policy Distillation for Continual GUI Agent Learning. *arXiv preprint arXiv:2607.04425*, 2026.
- G. Liu, J. Ye, P. Gao, W. Liu, J. Luan, Y. Liu, and Y. Li. SimuWoB: Simulating Real-World Mobile Apps for Fast and Faithful GUI Agent Benchmarking. *arXiv preprint arXiv:2605.25160*, 2026a.
- X. Liu, Y. Qin, et al. AutoGLM: Autonomous Foundation Agents for GUIs. *arXiv preprint arXiv:2411.00820*, 2024.
- Y. Liu, W. Xu, K. Huang, C. Chen, J. Zhao, P. Gao, W. Liu, J. Luan, S. Shang, B. Du, et al. CoME: Empowering Channel-of-Mobile-Experts with Informative Hybrid-Capabilities Reasoning. *arXiv preprint arXiv:2602.24142*, 2026b.
- Y. Liu et al. InfiGUI-R1: Advancing Multimodal GUI Agents from Reactive Actors to Deliberative Reasoners. *arXiv preprint arXiv:2504.14239*, 2025.
- Q. Lu, W. Shao, et al. GUI-Odyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. *arXiv preprint arXiv:2406.08451*, 2024.
- Z. Lu et al. UI-R1: Enhancing Efficient Action Prediction of GUI Agents by Reinforcement Learning. *arXiv preprint arXiv:2503.21620*, 2025.
- MAI-UI Team. Qwen-UI-Agent Technical Report: Real-World Centric Foundation GUI Agents. *arXiv preprint arXiv:2607.28227*, 2026. URL <https://arxiv.org/abs/2607.28227>.
- M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, et al. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. *arXiv preprint arXiv:2601.11868*, 2026.
- OpenAI. GPT-4o System Card, 2024. URL <https://openai.com/index/gpt-4o-system-card/>.
- OpenAI. GPT-5.5 System Card. <https://deploymentsafety.openai.com/gpt-5-5>, 2026.
- Y. Qin, Y. Ye, J. Fang, H. Wang, et al. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. *arXiv preprint arXiv:2501.12326*, 2025a.

- Y. Qin et al. MMBench-GUI: Hierarchical multi-platform evaluation framework for GUI agents. *arXiv preprint arXiv:2507.19448*, 2025b.
- Qwen Team and Data Team, Alibaba Group. QwenClawBench: Real-User-Distribution Benchmark for OpenClaw Agents. <https://github.com/SKYLENAGE-AI/QwenClawBench>, 2026.
- C. Rawles, Y. Li, et al. Android in the Wild: A Large-Scale Dataset for Android Device Control. *arXiv preprint arXiv:2307.10088*, 2023.
- C. Rawles, Y. Li, et al. AndroidWorld: A dynamic benchmarking environment for autonomous agents. In *International Conference on Learning Representations*, 2025.
- H. Shen, Z. Tang, S. Peng, L. Wu, A. Zhang, W. Wang, Y. Guo, C. Li, Z. Fang, Y. Ding, J. Li, F. Tang, Z. Ruan, Y. Zhang, X. Zhou, D. Yang, S. Fan, Z. Wan, H. Hu, X. Lai, P. Lyu, and C. Zhang. HyMobileAgent: Data-Environment Co-Scaling for Efficient GUI Agents. *arXiv preprint arXiv:2607.14548*, 2026. URL <https://arxiv.org/abs/2607.14548>.
- N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- Y. Sui, X. Huang, H. Li, F. Xu, J. Lv, H. Yan, Y. Shen, L. Liu, Z. Fan, Z. Meng, et al. AndroidDaily: A Verifiable Benchmark for Mobile GUI Agents on Real-World Closed-Source Applications. *arXiv preprint arXiv:2605.27761*, 2026.
- Z. Tang, Y. Liu, X. Lai, J. Li, P. Lyu, Y. Guo, Z. Fang, Y. Ding, Y. Zhang, W. Wang, et al. Phoneworld: Scaling phone-use agent environments. *arXiv preprint arXiv:2605.29486*, 2026.
- Tencent Hunyuan Team. PhoneBuddy: Training Open Models for Agentic Phone Use with Real and Mock App Environments. *arXiv preprint arXiv:2606.23049*, 2026. URL <https://arxiv.org/abs/2606.23049>.
- J. Wang, H. Xu, J. Ye, M. Huang, M. Yan, J. Zhang, F. Huang, et al. Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception. In *arXiv preprint arXiv:2401.16158*, 2024a.
- P. Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai, K. Chen, X. Liu, J. Wang, et al. Qwen2-VL: Enhancing Vision-Language Model’s Perception of the World at Any Resolution. *arXiv preprint arXiv:2409.12191*, 2024b.
- X. Wang, B. Wang, D. Lu, J. Yang, T. Xie, J. Wang, J. Deng, X. Guo, Y. Xu, C. Wu, et al. Opencua: Open foundations for computer-use agents. *Advances in Neural Information Processing Systems*, 38: 139756–139806, 2026a.
- Y. Wang, X. Chen, X. Jin, M. Wang, and L. Yang. OpenClaw-RL: Train Any Agent Simply by Talking. *arXiv preprint arXiv:2603.10165*, 2026b.
- Z. Wang, Z. Lin, J. Li, Y. Yi, Q. Gu, S. Dong, S. Wang, Z. He, M. Pang, Y. Chen, et al. MobileRL: Online Reinforcement Learning for Mobile Agents. *arXiv preprint arXiv:2510.14844*, 2025.
- D. Wu, R. Hao, H. Wang, S. Wu, H. Xiao, Z. Li, B. Zhou, Z. Ju, Z. Liu, L. Fan, et al. MobileGym: A Verifiable and Highly Parallel Simulation Platform for Mobile GUI Agent Research. *arXiv preprint arXiv:2605.26114*, 2026.
- Q. Wu, P. Gao, W. Liu, and J. Luan. BacktrackAgent: Enhancing GUI agent with error detection and backtracking mechanism. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 4250–4272, 2025.

- Z. Wu, Z. Liu, et al. OS-Atlas: A Foundation Action Model for Generalist GUI Agents. *arXiv preprint arXiv:2410.23218*, 2024.
- Y. Xia et al. GUI-R1: A Generalist R1-Style Vision-Language GUI Agent. *arXiv preprint arXiv:2504.10458*, 2025.
- J. Xiao et al. UI-Genie: A Self-Improving Approach for GUI Agents. In *Advances in Neural Information Processing Systems*, 2025. URL <https://arxiv.org/abs/2505.21496>.
- T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, 2024.
- H. Xu, X. Zhang, H. Liu, J. Wang, Z. Zhu, S. Zhou, X. Hu, F. Gao, J. Cao, Z. Wang, et al. Mobile-Agent-v3.5: Multi-platform Fundamental GUI Agents. *arXiv preprint arXiv:2602.16855*, 2026.
- W. Xu, Z. Jiang, Y. Liu, P. Gao, W. Liu, J. Luan, Y. Li, Y. Liu, B. Wang, and B. An. Mobile-bench-v2: A more realistic and comprehensive benchmark for vlm-based mobile agents. *arXiv preprint arXiv:2505.11891*, 2025. URL <https://arxiv.org/abs/2505.11891>.
- Y. Xu, X. Liu, et al. AndroidLab: Training and Systematic Benchmarking of Android Autonomous Agents. *arXiv preprint arXiv:2410.24024*, 2024.
- T. Xue, C. Peng, M. Huang, L. Guo, T. Han, H. Wang, J. Wang, X. Zhang, X. Yang, D. Zhao, et al. Evocua: Evolving computer use agents via learning from scalable synthetic experience. *arXiv preprint arXiv:2601.15876*, 2026.
- S. Yan et al. Step-GUI Technical Report. *arXiv preprint arXiv:2601.09661*, 2025.
- Y. Yang, D. Li, Y. Dai, Y. Yang, Z. Luo, Z. Zhao, Z. Hu, J. Huang, A. Saha, Z. Chen, et al. GTA1: GUI Test-time Scaling Agent. *arXiv preprint arXiv:2507.05791*, 2025a.
- Y. Yang, Z. Yang, Z.-Y. Dou, A. Nguyen, K. You, O. Attia, A. Szot, M. Feng, R. Ramrakhya, A. Toshev, et al. Ultracua: A foundation model for computer use agents with hybrid action. *arXiv preprint arXiv:2510.17790*, 2025b.
- Z. Yang, J. Zhang, et al. AppAgent: Multimodal Agents as Smartphone Users. In *arXiv preprint arXiv:2312.13771*, 2023.
- S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- B. Ye, R. Li, Q. Yang, Y. Liu, L. Yao, H. Lv, Z. Xie, C. An, L. Li, L. Kong, et al. Claw-Eval: Towards Trustworthy Evaluation of Autonomous Agents. *arXiv preprint arXiv:2604.06132*, 2026.
- J. Ye, X. Zhang, H. Xu, H. Liu, J. Wang, Z. Zhu, Z. Zheng, F. Gao, J. Cao, Z. Wu, et al. Mobile-Agent-v3: Fundamental Agents for GUI Automation. *arXiv preprint arXiv:2508.15144*, 2025.
- Z. Zhao, L. Zhang, Y. Liu, S. Chen, Y. Zhu, Y. Chen, J. Li, Z. Yang, K. Liu, C. Yuan, and Y. Huang. MobileBench-OL: Online real-device benchmarking for mobile agents. *arXiv preprint arXiv:2510.22014*, 2025.
- H. Zhou, X. Zhang, P. Tong, J. Zhang, L. Chen, Q. Kong, C. Cai, C. Liu, Y. Wang, J. Zhou, and S. Hoi. MAI-UI Technical Report: Real-World Centric Foundation GUI Agents. *arXiv preprint arXiv:2512.22047*, 2025. URL <https://arxiv.org/abs/2512.22047>.

- S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, et al. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.
- L. Zhu, J. Ding, J. Zhang, J. Xue, S. Liang, G. Zhang, X. Gao, Q. Gu, et al. Workflow-GYM: Towards Long-Horizon Evaluation of Computer-Use Agentic Tasks in Real-World Professional Fields. *arXiv preprint arXiv:2606.11042*, 2026.
- T. Y. Zhuo, V. M. Chien, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. GONG, J. Hoang, A. R. Zebaze, X. Hong, W.-D. Li, J. Kaddour, M. Xu, Z. Zhang, P. Yadav, N. Jain, A. Gu, Z. Cheng, J. Liu, Q. Liu, Z. Wang, D. Lo, B. Hui, N. Muennighoff, D. Fried, X. Du, H. de Vries, and L. V. Werra. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=YrycTj1lL0>.

A. Appendix



Figure 10 | Case. A trajectory of BlueLM-GUI comparing ride-hailing fares to Shenzhen MixC across DiDi and Amap.



Figure 11 | Case. An example of BlueLM-GUI completing “check my coupons” on Luckin Coffee.



Figure 12 | Case. A trajectory of BlueLM-GUI posting a Weibo with a custom hashtag and location.



Figure 13 | Case. A trajectory of BlueLM-GUI browsing upcoming movies on Douban ranked by popularity.



Figure 14 | Case. A trajectory of BlueLM-GUI comparing Starbucks delivery fees across Meituan and Taobao.